

A Decision Layer for Robot Object Search Under Unreliable Evidence, With a Checkable Provenance Record for Every Commitment

Kiran Nayudu DDS*

Independent researcher

Working paper v0.5 — September 2026

Abstract

A robot sent to find an object decides from evidence that arrives late, in pieces and sometimes false. We report a decision layer for that task in which every commitment carries a derivation record that a checker outside the engine replays. `robot_mind` is a deterministic engine over an append-only, bi-temporal evidence ledger with typed epistemic statuses. Beliefs are views over the ledger, each derived claim records the claims, rules and time constraints it rests on, actions are chosen by information gain, and a report is verified before it is trusted. No language model runs at inference time. On a seeded warehouse benchmark graded against hidden truth the engine is 100% correct at 1.75 actions per episode over 300 held-out episodes, and under budgets that brute force cannot exhaust it holds 98–100% at four and five actions where an exact belief-space planner reaches 73–90% and scripted policies 27–61%. Switching off multi-anchor hypotheses or repetition decay one at a time cuts the four-action figure from 98.4% to 72.8% and 87.6%, with no wrong commitment in either arm, so the margin is measured, not inferred. On a hospital floor with staged same-class decoys it is correct on 30/30 pre-registered seeds and fooled on 0/30, where class-matching policies are fooled on 16 and 26. On a hand-held phone camera’s footage of a lived-in house, with no instance identity and no association step, it commits correctly in 36/36 pre-registered hunts, follows 18/18 true tips and debunks 18/18 false ones. Two hosted LLM agents match its accuracy over 48 episodes at about 2,000 times its per-episode latency and without a replayable evidence-to-decision trail. A nine-suite stress battery finds it correct on all 1,860 unseen episodes and never confidently wrong under 320 planted lies per episode, and a mind kept across six days of nightly change holds 12/12 every day. Unchanged, the same build reaches 134/134 on ALFWorld’s unseen split, 95/96 fetches in AI2-THOR (83/96 with a real open-vocabulary detector in the sensing slot, all 83 commitments correct, identity from the simulator’s instance masks), 12/12 scored reports on DARPA SubT worlds and 8/8 on the hospital fetch benchmark, and as a ROS 2 lifecycle node it passes 20/20, 21/21 and 26/26 integration checks in simulation on a demo driver, stock Nav2 with SLAM Toolbox, and the detector under Nav2; one decision costs a third of a millisecond on a laptop core. Two boundaries were measured and each carries an acceptance test: per-decision cost still grows with lifetime memory after three increments cut it $\sim 7\times$, $4\times$ and $\sim 3\times$, and at a raised resolve bar the engine names a room for an absent object in 121/300 episodes because its verdict vocabulary has no justified not-found. The benchmark kit, every baseline, the proof checker and per-episode ledger exports are released: every baseline regenerates byte for byte, and the checker re-validates the provenance of every proof in this

*Code and data availability: the benchmark kit (scenario generator, every scripted and belief-space baseline, the LLM comparison harness, the independent proof checker, and every engine trace and artifact this paper reports; Apache-2.0, data CC BY 4.0) and a demonstration build of the engine are released at <https://github.com/fourwheels2512/unknown-goals> with this paper’s deposit. The engine’s source is not released; its runs ship as checkable traces (see the reproducibility statement). Development disclosure: the system was designed and implemented by an LLM assistant under the author’s direction; the benchmarked artifact runs no language model (§6, item 7).

paper with no engine code. The checker certifies provenance, not the belief arithmetic, which is withheld together with the engine’s source; the engine ships as a demonstration build.

1 Introduction

A robot sent to find something decides from evidence that arrives late, in pieces and sometimes false: a camera sighting at low confidence, a badge event at a door, a report that the cart took it. The decision that matters is the commitment, “the object is in room X ”, because a wrong one costs a trip, a mission or a safety case. Three kinds of agent make that decision today, and none can show afterwards why it committed. LLM agents reason fluently over the evidence but keep no separation between what was observed and what was inferred. Learned navigation policies need demonstrations or reward and carry no provenance. Classical belief-space search keeps a belief but not the evidence behind it: no epistemic status, no supersession, no defence against a planted claim, no proof. This paper reports a decision layer that derives every commitment from the evidence, attaches a derivation record that a separate checker replays, and was graded against ground truth in every environment it entered, with one build throughout.

The task is object search, the kernel of object-goal navigation in embodied AI. A tool goes missing in a simulated warehouse, and evidence arrives as noisy, timestamped sensor claims (camera sightings, badge events, cart-tracker reports). The agent chooses physical checks, such as inspecting a location or scanning a cart, to find the object within an action budget. The benchmark abstracts perception and locomotion so that the search policy is the variable under test; §4.4 removes each abstraction in turn with the engine unchanged.

The engine, `robot_mind`, is a deterministic program with no language model at inference time. Claims enter an append-only, bi-temporal evidence ledger with typed epistemic statuses, so a hypothesis can never become an observation by scoring high and a stale report can never overwrite a newer one. Beliefs are views over the ledger, not mutable state. Every derived claim records the claims, rules and time constraints it rests on, and a checker outside the engine replays that record. The planner chooses each action for the information it buys, a fresh report is verified before it is trusted, and a refuted belief widens the search instead of ending it. From one integer seed the worlds, observations, choices and proofs reproduce byte for byte. The thesis is this: *an append-only evidence ledger with explicit epistemic statuses, plus a verify-then-redirect search policy over it, gives a low-latency, closed-domain object-search agent whose every decision carries a machine-checkable provenance trail, and that property, unlike accuracy, was supplied by none of the LLM agents evaluated here at any scale we tested.*

The thesis held everywhere it was tested. On the warehouse benchmark the engine is 100% correct at 1.75 actions per episode. Where brute force cannot finish it keeps 98–100% of episodes at four and five actions, against 73–90% for an exact belief-space planner over the same evidence and 27–61% for scripted policies. Two hosted LLM agents reach the same accuracy at about 2,000 times the latency and with no trail a checker can replay. The same build then left the warehouse unchanged: 134/134 on ALFWorld’s unseen split, 95/96 fetches in AI2-THOR, 12/12 scored reports on DARPA SubT worlds, 8/8 on a hospital floor, and every target on a Mars terrain and a seabed. On that floor it was correct on 30/30 pre-registered decoy seeds and fooled on 0/30. It then crossed the robot’s seams with real components in the slot. An open-vocabulary detector fetched 83/96 with every commitment correct. Stock Nav2 with SLAM Toolbox in simulation passed 21/21 integration checks, and 26/26 with the detector in the loop. A hand-held phone camera in a lived-in house gave 36/36 hunts, 18/18 true tips followed and 18/18 false tips debunked, all pre-registered. A mind kept across six days of nightly change holds 12/12 every day. A nine-suite stress battery finds it correct on all 1,860 unseen episodes and never confidently wrong under 320 planted lies. One decision costs a third of a millisecond on a laptop core.

1.1 What was uncertain at the outset

Six of the eight technical uncertainties open at the outset were resolved in the engine’s favour, three of them only after a benchmark had exposed a failure and the fix had been built. The last two were measured to a boundary, and each boundary now carries an objective with an acceptance test (§8). None of this was assured when the work began. To our knowledge no prior object-search system attaches a replayable derivation to its commitments, and the closest work (§7) either stores memory without closing the hypothesise–act–verify loop or plans over beliefs without keeping the evidence. Development was therefore organised around the ways the engine could fail. Table 1 lists the eight, how each was tested and what came out. The method was the same each time: state the hypothesis, build the mechanism, grade it against hidden truth, submit the result to adversarial review, fix, and re-verify every earlier battery byte for byte before keeping the fix.

The contributions are:

1. the decision layer (§2): an evidence ledger with typed statuses and proof paths, an information-gain planner that verifies before it trusts, and three mechanisms found by benchmarking, deterministic from one seed and released as a demonstration build;
2. a benchmark discipline (§3): hidden-truth grading, a declared seed split for every evaluation, a fairness contract for LLM agents, scripted and exact belief-space baselines under the identical episode contract, and pre-registration wherever the world is not seeded;
3. the measurements (§4): efficiency and necessity under a sub-exhaustive budget, accuracy parity with two hosted LLM agents at three orders of magnitude lower latency, one build unchanged across ALFWorld, AI2-THOR, DARPA SubT worlds, a hospital floor, a Mars terrain and a seabed, the robot’s three seams crossed with real components, a persistent mind at measured cost, a nine-suite stress battery, and the first measurement of the commitment discipline against an absent object;
4. a benchmark kit in which every baseline regenerates byte for byte and every engine proof re-validates from its ledger export with no engine code, and a research programme with an acceptance test per objective (§8).

§2 describes the decision layer. §3 states how every number in the paper was produced. §4 reports the results in the order of the uncertainties. §5 discusses what they establish, §6 bounds them, §7 places the work beside its neighbours, §8 sets out the research that follows and §9 concludes. The kit and the artifacts are described after the conclusion. Appendix A is the ROS 2 integration note, Appendix B the development record and Appendix C a glossary of the acronyms used.

2 The decision layer

The engine is one deterministic Python application of ten modules (74 source files, inventoried in Appendix B), and Figure 1 is the loop it runs on every decision. This section walks that loop once, states the design rules it implements, and names the three mechanisms benchmarking added to it.

A claim arrives typed and stasured. Every input enters the SQLite evidence ledger as an append-only record carrying a subject, a predicate and an object, two clocks (event time and ingestion time), a source, a confidence and an epistemic status. The status says what kind of thing the record is: OBSERVED for what a sensor saw, REPORTED for what somebody said, DERIVED for what the engine concluded, and seven more. A hypothesis therefore never becomes an observed fact by scoring high, and a status change appends history rather than touching the raw row, so a late-arriving stale claim cannot overwrite newer knowledge. One integer seed reproduces the worlds, observations, choices, proofs and learned-rule metrics of a whole campaign byte for byte.

uncertainty	how it was tested	outcome	where
1. Sufficiency. Can a compact deterministic engine, with no model in the loop, match LLM agents on temporal, identity and causal reasoning from unreliable sensors?	warehouse benchmark graded against hidden truth; head-to-head with Gemini 3.7 Flash and DeepSeek chat (August 2026) under the fairness contract, latency measured end to end through their APIs	yes: accuracy parity with two hosted LLM agents at three orders of magnitude lower latency, and a replayable evidence-to-decision trail that neither agent produced under this harness. The multi-anchor hypotheses that secured it (§2) were found by benchmarking, after one false low-confidence sighting had severed the true carrier chain: 52% success at the hardest level before, 92% after	§4.1–4.2
2. Necessity. Does the evidence machinery buy anything a for-loop does not?	budget sweep against an exact planner over the plain filter’s belief and against scripted policies	yes, below the exhaustive budget: 98–100% at four and five actions against 73–90% for the exact planner and 27–61% for the scripted policies, and 70–90% at two and three against 41–53% and 11–34%. At the full budget every policy scores 100%, so success under a budget is the discriminating quantity. No policy commits wrongly in the sweep’s 18,000 episodes, because success requires a confirming inspection. A leave-one-out ablation on the shipped build attributes the margin: multi-anchor hypotheses and repetition decay each carry a disjoint share, transport hand-off’s share is within noise	§4.1
3. Deception. Does evidence hygiene survive an adversary that plants confident false claims?	the gaslight level (L6); floods of planted lies	yes: 40/40 under gaslight, and never confidently wrong under up to 320 planted lies per episode	§4.7
4. Lifelong memory. Is a memory that never forgets an asset in a changing world?	a persistent mind kept across days in a changing AI2-THOR world	yes, with two organs built for it: age-decayed evidence and refutation-driven search hold the same mind at 12/12 every day across six days of nightly change. Before them it fell from 12/12 on day 1 to 4, 1, 0, 2 and 0 of 12 on days 2–6 (2–6 of 12 with drives on)	§4.5
5. Perception seam. Do commitments derived from a real detector’s raw output, with no instance identities and no association step, stay correct?	pre-registered hunts on hand-held phone footage of one house	yes: 36/36 pre-registered hunts, 18/18 true tips followed and 18/18 false tips debunked. Three graded sittings (12/36, 29/36, 33/36) diagnosed and fixed a resolve-on-sighting rule that was right wherever an inspection could confirm a sighting and wrong when the same clip returned the same detection	§4.4
6. Navigation seam. Do the decisions survive a navigation stack’s timeouts, aborts and drift?	stock Nav2 with SLAM Toolbox in simulation, then with the detector in the loop as well	yes: 21/21 integration checks on stock Nav2 with SLAM Toolbox and 26/26 with the detector in the loop; the four failed bring-up runs are on record with their fixes in the simulated body and the stack’s configuration	§4.4
7. Cost of memory. Can per-decision cost stay bounded as the ledger grows without end?	a 300-day life; per-decision latency measured across three increments of incremental view maintenance	bounded in three increments, $\sim 7\times$, $4\times$ and $\sim 3\times$, every decision byte-identical; the remaining growth has a named cause and a bounded-cost objective with an acceptance test (O2, limitation 11)	§4.5
8. Absent object. Does the commitment discipline hold when the object is not in the building at all?	target removed from the world; raised resolve bar	measured to its boundary: the rule that closes an exhausted search names a room in 121/300 absent-object episodes at a raised bar, because the verdict vocabulary has no justified not-found. That verdict is specified, and this battery is its acceptance test (O1, limitation 13)	§4.6

Table 1: The eight technical uncertainties open at the outset: how each was tested against hidden ground truth, what came out, and where it is reported.

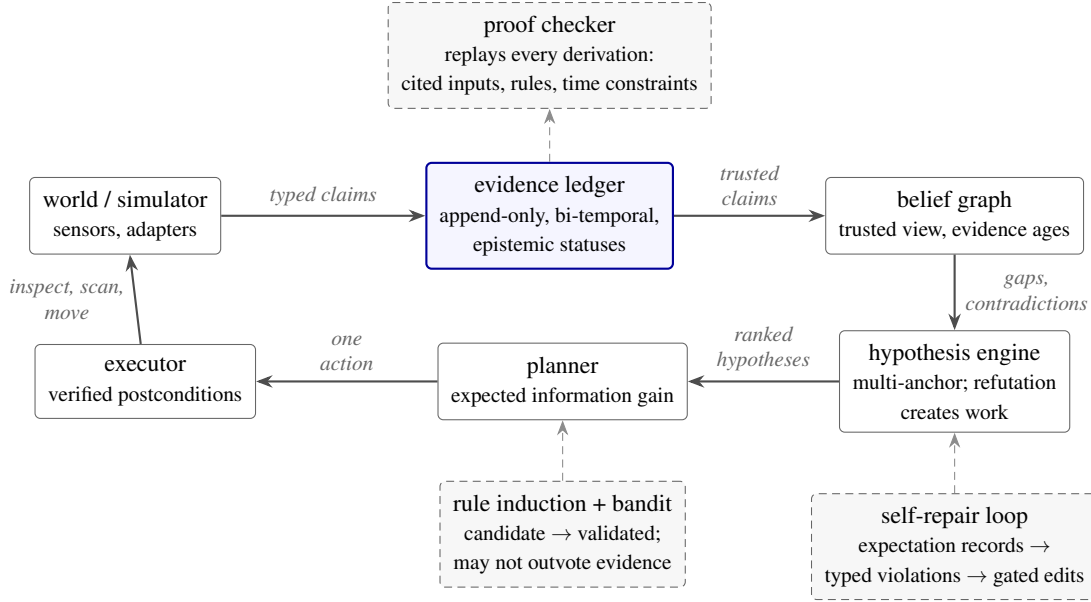


Figure 1: The reasoning loop. Solid boxes run on every decision; dashed satellites audit and tune it. Claims enter typed and stasured, beliefs are views over the ledger rather than mutable state, and every derived claim can be replayed by the proof checker. The ledger and its belief-graph view together form an explicit, queryable world model.

Beliefs are views over that ledger rather than mutable state. Each belief is a deterministic function of ledger rows, normalized per question, and evidence ages: an old sighting and an old absence both fade, while validated rules never decay. Every derived claim records the claim ids, rule ids, assumptions and time constraints it rests on, and a checker outside the engine replays that record against the ledger. The benchmark reports the resulting validity rate, 1.000 in every run reported here.

Gaps and refutations both create work. A missing variable becomes ranked hypotheses and an information-seeking action, never a terminal “cannot determine”. Repeated failed checks eventually widen the search to places the evidence never pointed at, and a single absence does not, because one absence still reads as possible occlusion. Without that widening an object moved overnight is unfindable, which is the failure the dynamic-house protocol of §4.5 exposed before the mechanism was built. The planner then picks one action for the information it buys, over the belief the ledger defines.

The executor carries that action out and verifies its postconditions, so a fresh report is checked before it is trusted rather than after it has been acted on. Identity grounding is weighted and keeps ambiguity below a threshold instead of resolving it. The learned layer, rule induction and a contextual bandit, may reorder the candidates the planner already justifies and may never outvote evidence or create an observed fact. §4.7 describes the self-repair loop that audits the whole of it.

The language-model seam. A language model has one designed seam, a CognitiveProposer protocol, and the shipped implementation is a deterministic template with no model dependency. The seam allows hybrid operation: a model may translate natural requests into structured goals and propose up to two additional hypotheses per reasoning round. Proposals pass an engine-side verification gate and share a bounded belief mass (8%), so they can steer attention but never outvote evidence, touch the ledger, or alter proofs. No language model runs in any measurement in this paper.

The design rules, for reference. Seven rules state the loop above in the engine’s own terms.

1. **One canonical evidence store.** Claims are append-only records carrying subject, predicate and object, *two clocks* (event time and ingestion time), source, confidence and epistemic status. Status changes append history and raw rows are never touched, so late-arriving stale data cannot overwrite newer knowledge.
2. **Epistemic statuses.** OBSERVED / REPORTED / DERIVED / HYPOTHESIZED / PREDICTED / SIMULATED / VALIDATED_RULE / REJECTED / SUPERSEDED / CONTRADICTED. A hypothesis never becomes an observed fact by scoring high; only a new observation carries OBSERVED.
3. **Proof paths, independently checked.** Every derived claim records its input claim ids, rule ids, assumptions and time constraints. A separate checker replays every proof against the ledger, and the benchmark reports the validity rate (1.000 in all reported runs).
4. **Unknown creates work.** A missing variable becomes ranked hypotheses and an information-seeking action, never a terminal “cannot determine.”
5. **Explainable belief arithmetic.** Every belief is a deterministic function of ledger rows, normalized per question, and the engine’s developer output prints every number with its full breakdown (the demonstration build prints belief and status only). Evidence *ages*, symmetrically for support and contradictions, while validated rules (semantic memory) never decay. How support, contradiction, age and rule reliability combine, and the constants they combine under, are withheld from this version of the paper (see the reproducibility statement).
6. **Refutation creates work, exactly like ignorance.** Repeated failed checks eventually widen the search to places the evidence never pointed at; a single absence does not, because it still reads as possible occlusion. The trigger and the widening rule are withheld from this version of the paper. Without this mechanism an object moved overnight is unfindable; the dynamic-house protocol (§4.5) exposed that failure, and the mechanism was built in response.
7. **Determinism.** A single integer seed reproduces worlds, observations, choices, proofs and learned-rule metrics byte-for-byte.

What benchmarking added. Three mechanisms proved load-bearing and were *discovered through benchmarking*, not designed up front: **multi-anchor hypotheses** (the most recent sighting is not trusted blindly; adding them fixed a failure mode where one false low-confidence sighting severed the true carrier chain and lifted hardest-level success 52%→92%); **transport handoff** (a container that leaves with plausible cargo changes what its parent hypothesis is worth: one object cannot be in two places); and **repetition decay with a policy value gate** (repeating an action without new evidence costs it value, and the learned policy may only choose among candidates the planner still justifies). How each of the three is computed is withheld from this version of the paper; what each does to the numbers is measured in §4.1.

Their measured shares are in §4.1, under the evaluation discipline the next section states.

3 Evaluation method

Every number in this paper was produced under one discipline: grading against hidden truth the agent never sees, a seed or world split declared per evaluation, and an adapter that may not read the answer. This section states that discipline once, so the results can be read without it.

Terms for verdicts, and what the checker certifies. In this paper *commitment* names one event, and every count that carries the central invariant counts that event. Table 2 fixes the vocabulary; §4.6 reports each row’s rate under the one condition that exercises them all. Four thresholds recur. The *resolve bar* (*sighting_resolves_at*: 0.0 in the warehouse and simulator batteries, 0.9 in the real-pixels harness and in §4.6) is the sighting confidence at which the direct path commits. The *claim gate* (0.05) is the detector confidence below which a detection never becomes a claim. The *confirmation confidence* (0.9) is what an inspection writes when it confirms a sighting. The *sighting bar* of §4.4 is the resolve bar under that harness’s name.

term	meaning	triggers an action	retractable	a wrong commitment if wrong
hypothesis	a ranked candidate holder or room in the beam	yes: actions discriminate among them	yes	no
transient verdict	a resolved location held between two reasoning calls and re-examined at the next; it triggers the inspection that confirms or retracts it	yes	yes	no; reported separately (§4.6: 74/300 present-target episodes at bar 0.9; 799 firings in the absent arm)
committed verdict (FOUND)	the resolved location when the episode ends: in the warehouse and the simulators only after a confirming inspection; in the real-pixels harness on a sighting that clears the resolve bar, or by exhaustion once every zone has been inspected	yes	only by a later episode	yes: every “wrong commitment” count in this paper is this event
no verdict	the episode ends with no resolved location (budget spent, or the resolving sighting aged past the direct path’s window); today indistinguishable from a judged absence	no	—	no; reported as a rate (§4.6: 104/300 in the length-matched control, 179/300 in the absent arm)
NOT-FOUND (proposed, §8 O1)	a refusal carrying a coverage certificate over the rooms	yes	after a movement event touching the target	yes, if the object was present
proof path	the derivation record behind a verdict: the cited claims, rule ids and time constraints. The kit’s checker certifies <i>provenance integrity</i> (every cited claim and rule exists and the time constraints hold) and checks <i>logical consistency</i> structurally (rule ids and arity), not by re-deriving the conclusion; <i>empirical correctness</i> is measured against hidden truth and never inferred from validity (§4.1)	—	—	—

Table 2: The verdict vocabulary. Every “zero wrong commitments” figure in this paper counts the third row; the second and fourth rows are reported as their own rates where they occur.

The generator and the levels. A seeded generator produces warehouse scenarios in two separated layers. The *hidden truth* (actual object locations and causal movement chains) is seen only by the evaluator. The *observable evidence* (noisy, incomplete, sometimes contradictory claims) is all the agent sees. Five difficulty levels, L1 to L5, run from a direct sighting to contradictions, occlusion and wrong-branch recovery; the notation paragraph below names each. Truth branches are sampled. At L3 and above the cart transports the object in $\sim 72\%$ of episodes, the person keeps it in $\sim 20\%$, and the absence report is spurious in $\sim 8\%$. No

fixed strategy is always right, and grading is against ground truth, not plausibility.

Seed and world discipline. Table 3 lists the evaluations of §4 in the order they are reported and states, for each, which seeds or worlds were available during design and debugging, which were used only for reporting, and which were declared before they ran. Where a fix followed a first run on a reporting set, the result says so. The primary benchmark’s held-out and comparison sets are disjoint seeds from the same generator. A held-out generator family with different structural parameters (location count, carrier count, report reliability, delays) is future work.

The fairness contract for LLM agents. All agents run under one fairness contract: identical evidence text, action menu, simulator, sensors, budget (8 actions) and success rule, graded against identical hidden truth. Temperature is 0, with one re-prompt permitted on unparseable output, over 48 episodes (12 seeds $\times$ levels 2–5). Local models run through Ollama [36, 37]. The API models are deepseek-chat [15] and gemini-3.7-flash [19], both accessed August 2026 (hosted models drift; the artifacts record the behavior measured at that date).

The embodied adapter contract. Every embodied evaluation shares one contract. The engine is byte-identical in all of them and decides every search step from typed claims on an initially empty ledger. A minimal adapter translates simulator perception into claims and engine actions into commands, and may not read target placements. Success is graded by the simulator’s own signals or by the benchmark’s published rule. Locomotion is teleport-abstracted, as ALFWorld’s `go to` is, and each adapter’s sensing model is stated with its result, so what the embodied numbers measure is the search policy rather than a full robotics stack.

Three further boundaries hold across all of them. There is no SLAM except in the Nav2 lanes, where SLAM Toolbox localizes the base while the frustum sensor’s vantage is read from physics. Sensing is simulator metadata except in three places: the AI2-THOR detector battery and the camera lane use tracker stand-ins for instance association, and the phone battery has a real camera and no association step. MoveIt 2 was not run, and its interface is the grasp-pose hand-off, a `PoseStamped` in the goal frame. AI2-THOR’s placement randomization is not bit-stable across runs, so those episodes are artifact-logged rather than seed-reproducible. Two further simulators were not run: Habitat’s headless builds need an EGL device that WSL2 does not expose, and OmniGibson’s Isaac Sim installation was not completed. `docs/EMBODIED.md` records the state of each and the commands that resume them.

Which figure is the headline. Unless stated otherwise a step figure is the shipped build at the eight-action budget with its learned rule loaded. The headline efficiency figure is the held-out battery: 300 episodes, seeds 10000–10074, levels 2–5, learned layer off, 100% correct at 1.75 actions. Every other step figure names its seed family, levels, build and learned-layer state where it appears.

Notation, level names and version tags. Success rates carry Wilson 95% intervals (the score interval for a binomial proportion). Benchmark levels: L1 direct sighting; L2 person-carrier bridge; L3 person-and-cart transport chain; L4 distractors, decoys and stale claims; L5 contradictions, occlusion and wrong-branch recovery; L6 the adversarial planted-report level (“gaslight”). Version tags name engine milestones: v0.2.0 is the config version of the rule set after the temporal probes (limitation 4); v0.3 added evidence ageing and the persistent-mind organs; v0.4 closed the metacognitive loop (§4.7); v0.5 is the build reported here. The engine is written `robot_mind` throughout, and Appendix C defines the acronyms.

evaluation	development (design, debugging)	reporting only	declared before running
warehouse engine, learned layer	seeds 0–249 (rules designed on this distribution; rule mined here)	held-out 10000–10074 $\times$ L2–5, 300 episodes per arm	—
warehouse baselines, LLM contract, budget sweep	— (engine changes after the first campaign came from hand-built temporal probes; re-run figures reported beside originals)	seeds 20000–20011 $\times$ L2–6	—
ALFWorld	one belief-hygiene fix after the first run; first run kept in history	valid_unseen 134 games; random baseline on both splits, sweep on valid_seen	external benchmark
AI2-THOR, oracle sensing	seeds 7, 11, 23, 42, 99, 256 (adapter fix loop)	seeds 137, 314 (no fix diagnosed on them)	—
DARPA SubT	bridge fixed on tunnel_01 first, then the other worlds	4 worlds $\times$ seeds 7, 11, 23	external worlds and scoring rule
hospital fetch, tips, decoys	sensor-model fix diagnosed on the seed-7 smoke; battery re-run	seeds 7, 11, 23, 42, 99, 137, 256, 314 $\times$ 3 policies; decoys on the six false-tip seeds	tip-coin addendum seeds 3, 5, 6, 8 chosen by coin alone before running; 30-seed decoy power run pre-registered (first 30 seeds with a false tip coin; run to completion: engine 30/30, 0/30 fooled)
AI2-THOR, real detector	seeds 7, 11, 23 (synonym table, gate, confirmation ordering)	—	seeds 42, 99, 137, 256, 314 declared and launched on commit 8b4a83f before any ran: 52/60, 52/52 correct; two start-up crashes rerun as fresh draws, crash records shipped
ROS 2 stack, Nav2, camera lane	four diagnostic Nav2 runs, resolved in the simulated body and in configuration; one CPU smoke of the camera projection on teleported views	staged truths bay_3/1/4/2 + one tip, three lanes (demo driver, Nav2, Nav2 + YOLO-World)	—
AI2-THOR, lifelong	habit spot chosen by sweep; first curve shipped	the reported 30-episode curve, 4-episode wrong-habit run	—
stress battery, self-repair exam	v1 sabotage roster screened for detectability	sealed seeds and hidden-truth grading outside the loop’s reach	exam v2 jitters every field before sabotage

Table 3: What was available to whom, per evaluation. “Reporting only” sets were not used to take design decisions; exceptions are stated in the development column and at the result.

4 Results

The subsections below take the uncertainties of Table 1 in order, each opening with the question it settles and the answer it reached. §4.3 is the exception. It reports what the same build did in worlds it was not built for, which no uncertainty in the table anticipated.

4.1 Does the evidence machinery matter?

Uncertainties 1 and 2 asked whether a compact deterministic engine is sufficient for this task and whether its evidence machinery buys anything a for-loop does not. On the warehouse benchmark the answer to both is yes, and the discriminating measurement is efficiency under a budget brute force cannot exhaust.

Held-out result and the learned layer. The engine finds every target in 300 held-out episodes per arm (seeds 10000–10074 $\times$ levels 2–5, disjoint from training). Its learned layer cuts actions to success by 16% overall and by 19% on the hardest level (Table 4). The layer is a transport rule mined from the engine’s own episodes and a contextual bandit over action types, evaluated frozen. The rule deploys at its measured reliability of 0.71, and proof paths cite it at that value. Two caveats apply. The proof checker validates provenance only: cited claims and rules exist in the ledger and the time constraints hold on replay. It does not re-derive conclusions, and a valid trail never says the inputs were true. The miner’s labels are simulator ground truth; a physical deployment would need a labeling process.

metric	baseline	learned layer
task success	1.000	1.000
avg. actions to success	1.75	1.47 (−16%)
L5 actions	2.33	1.88 (−19%)
proof validity	1.000	1.000

Table 4: The learned layer keeps success and proof validity at 1.000 and cuts actions to success by 16% overall and 19% on L5 (300 held-out episodes per arm).

Scripted baselines at the full budget. Four scripted policies, named in Table 5 and defined in Appendix B, run levels 2–6 on the head-to-head seeds under the LLM episode contract (`experiments/scripted_baselines.py`). Every policy reaches 60/60, because eight actions cover the six locations and two carts. Success does not discriminate at this budget, as on ALFWorld (§4.3); steps and the cost of deception do. A plain Bayesian belief filter, with argmax-posterior selection and none of the engine’s machinery, needs 4.80 steps (4.40 on L2–5), 0.12 fewer than the random walk. The engine’s 1.75 is 2.5 $\times$ fewer still, so the efficiency comes from multi-anchor carrier chains, transport handoff and contradiction bookkeeping, not from belief updating in general. On the gaslight level the evidence-following heuristic chases the planted tip (6.92 steps) and the filter trusts the same tip (6.42). The evidence-ignoring sweep is unlucky in its fixed order (6.83). The random order pays 3.92, the smallest margin in the row, because it cannot be deceived. The engine pays 3.25: it verifies the tip once and redirects.

Below the exhaustive budget. Where brute force cannot finish, the benchmark discriminates. Table 6 and Figure 2 cut the action budget from eight to two for every policy (`experiments/budget_sweep.py`), 500 episodes per cell over 100 seeds per level, with Wilson 95% intervals. At five actions, one fewer than the six locations, the scripted policies reach 34–61% and the engine stays at 100% [99.2, 100]. At four they

policy	correct (L2–6)	avg. steps	gaslight L6 steps
sweep (ignores evidence)	60/60	5.88	6.83
random order	60/60	4.92	3.92
last-seen-first	60/60	5.97	6.92
belief filter (no ledger/statuses/proofs)	60/60	4.80	6.42
belief-space planner, exact (filter’s belief + lookahead)	60/60	3.13 (2.60 on L2–5)	5.25
engine	60/60	2.05 (L2–6; 1.75 on L2–5)	3.25

Table 5: Scripted baselines under the identical episode contract (12 seeds/level). Every policy reaches 100% because the 8-action budget covers the 6 locations + 2 carts, so success does not discriminate here, as on ALFWorld; the discriminating columns are steps (engine 1.75 on L2–5 against the exact planner’s 2.60 and the belief filter’s 4.40) and the cost of deception (engine 3.25; the belief filter, exact planner and evidence-following heuristic 5.25–6.92).

reach 27–48% against the engine’s 98.4% [96.9, 99.2]. At three the engine still finds 90% against 20–42%. No policy commits wrongly in any of the 18,000 episodes, because the success rule requires a confirming inspection. The count is 15,000 in the budget sweep, of which the engine ran 3,000, and 3,000 in the planner sweep.

The strongest scripted competitor is an exact belief-space planner under the same contract (`experiments/belief_planner.py`). It maximizes the probability of finding the target within the remaining budget, looking ahead over what each check could reveal (solved to optimality; recursion and sensor constants in Appendix B). It starts from the belief filter’s own reading of the evidence, with the true sensor model. So it is optimal for the POMDP the filter’s model defines, and lookahead is the only thing it adds. Lookahead adds a lot: 73.4% [69.4, 77.1] at four actions and 90.0% [87.1, 92.3] at five, against the filter’s 42.6%. It does not reach the engine’s 98.4% and 100% at the same budgets, or its 2.05 actions at eight. Lookahead closes about half of the filter’s gap to the engine. What remains is the choice of belief the lookahead runs over: multi-anchor hypotheses, transport handoff, contradiction bookkeeping and verify-then-redirect.

Each mechanism’s share. A leave-one-out ablation on the shipped build measures what the three mechanisms of §2 contribute (`experiments/ablation_loo.py`; `data/artifacts/ablation_loo.json`). Each is switched off in turn and the engine runs the same 3,000 episodes as the sweep, learned rule loaded and every proof checked. The shipped arm reproduces every cell of Table 6. Without multi-anchor hypotheses the engine keeps 70.0% of episodes at three actions, 72.8% at four and 74.0% at five, against 89.8%, 98.4% and 100%, and it does not recover at the full budget: 78.8% at eight actions, where every scripted policy reaches 100%. Without repetition decay it keeps 80.6%, 87.6% and 89.0%, and 89.2% at eight. Both arms’ intervals are disjoint from the shipped arm’s at every budget from three upward. Without transport hand-off it keeps 89.0%, 98.0% and 99.8%, inside the shipped arm’s interval at every budget, so that mechanism’s share is not measurable on this family. No arm commits wrongly in any of its 3,000 episodes; every point of margin lost is a no-verdict. Multi-anchor hypotheses and repetition decay carry the sub-exhaustive margin, and the necessity claim rests on them.

The engine’s step counts differ by build and seed family. The 1.75 (L2–5) and 2.05 (L2–6) of Table 5 are the original campaign build on the 12 head-to-head seeds. The current build with its learned rule loaded gives 1.83 and 2.10 on the same seeds, and 1.80 and 2.05 over the sweep’s 100 seeds. The 1.47 of Table 4 is the learned layer on the 300 held-out episodes per arm (seeds 10000–10074, levels 2–5).

Under a budget that brute force cannot exhaust, and against these baselines on this generator, the machinery is necessary as well as efficient. A robot with a finite battery operates in that regime.

budget	engine	planner (exact)	belief filter	random	last-seen	sweep
2	69.6 ± 4.1 (1.63)	40.6 ± 4.3 (1.67)	40.6 ± 4.3 (1.67)	23.8 ± 3.7 (1.88)	11.0 ± 2.8 (1.96)	14.0 ± 3.1 (1.93)
3	89.8 ± 2.7 (1.93)	52.8 ± 4.4 (2.26)	42.0 ± 4.3 (2.26)	34.2 ± 4.2 (2.64)	19.6 ± 3.5 (2.85)	21.0 ± 3.6 (2.79)
4	98.4 ± 1.2 (2.03)	73.4 ± 3.9 (2.74)	42.6 ± 4.3 (2.84)	48.2 ± 4.4 (3.30)	27.4 ± 3.9 (3.65)	28.6 ± 4.0 (3.58)
5	100 [99.2, 100] (2.05)	90.0 ± 2.6 (3.00)	42.6 ± 4.3 (3.42)	61.0 ± 4.3 (3.81)	34.0 ± 4.2 (4.38)	35.6 ± 4.2 (4.30)
6	100 [99.2, 100] (2.05)	97.8 ± 1.4 (3.10)	44.2 ± 4.4 (3.99)	72.2 ± 3.9 (4.20)	42.6 ± 4.3 (5.04)	42.6 ± 4.3 (4.94)
8 (shipped)	100 (2.05)	100 (3.12)	100 (4.80)	100 (4.62)	100 (5.88)	100 (5.78)

Table 6: Correct-commitment rate in % with the Wilson 95% half-width (mean actions in parentheses), 500 episodes per cell (100 seeds $\times$ levels 2–6, seeds 20000–20099), as the action budget shrinks; at 100% the interval is one-sided and is printed as its bounds. The planner is the exact finite-horizon belief-space planner of §4.1 (the filter’s reading of the evidence, the true sensor model, optimal lookahead). Engine on the current build with its learned rule loaded: 1.83 on L2–5 and 2.10 on L2–6 at budget 8 for the 12-seed subset, 1.80 and 2.05 for the 100 seeds; the 1.75 and 2.05 of Table 5 are the original campaign build. Zero wrong commitments in every cell. `data/artifacts/budget_sweep_100.json`, `belief_planner_sweep.json`; the 12-seed runs that match Table 5 at budget 8 are `budget_sweep.json` and `belief_planner_sweep_12.json`.

Confidence and counterfactuals. The engine’s stated beliefs are calibrated, and every answer is attributable to named claims. Calibration against hidden truth gives ECE 0.047 on levels 1–5 and 0.065 under gaslighting, and beliefs above 0.7 are right 100% of the time (Figure 3). The `robot-mind whatif` command re-computes beliefs with a claim subset removed: on gaslight episodes the planted tip accounts for 100% of the belief in the fake location, and removing it leaves the true chain’s ranking unchanged. Appendix B gives the calibration methodology.

Accuracy and efficiency against scripted opponents settle necessity on this generator. The next question is what the same evidence buys against an agent that reasons in language.

4.2 Against LLM agents

Uncertainty 1 asked whether the engine could match LLM agents on temporal, identity and causal reasoning from unreliable sensors. Over 48 episodes under the fairness contract of §3 it does, at about three orders of magnitude lower latency, and it is the only agent in the comparison that leaves a trail a checker can replay.

The engine’s ~ 5 ms is the campaign’s per-episode wall-clock on an unpinned machine. The pinned-core profile of §4.7 gives 2.8–3.1 ms median and 4.8–5.4 ms p95 per episode. The LLM figures are end-to-end wall-clock through the provider’s API, with network, queueing and the tool loop included. The ratio is therefore about 2,000 $\times$ per episode, not a hardware-normalised inference benchmark.

The strong scaffold coaches the 7B model in the engine’s own strategy, to control for prompting. It does not change the result: 39/48 against 40/48, one episode, inside binomial noise at $n = 48$. Knowing the strategy is not the bottleneck at this model scale; holding calibrated belief state across steps is.

Between 8B and the two hosted models the measured gap vanishes. Both `gemini-3.7-flash` and `deepseek-chat` go 48/48, and Gemini stays perfect under the L6 gaslighting campaign (12/12 at 2.50 steps against the engine’s 3.25 on the same seeds). By the same noise standard the 45/48-versus-48/48 comparison is not significant at $n = 48$ (Fisher exact $p = 0.24$). The precise claim is that no accuracy gap is *detectable* for the two hosted agents at this sample size. The step difference is the engine’s verify-before-trusting discipline: one inspection that disproves the planted tip, recorded on the ledger. Gemini ignores the tip on judgment, correctly, and cannot afterwards show why that was safe.

The gaslight campaign (L6) adds an adversary who knows the true final location and plants a confident fake sighting steering away from it, with further moves listed in Appendix B. Over 40 seeded episodes the engine goes **40/40** with 100% proof validity at 3.1 actions on average, against ~ 1.9 undeceived on the same

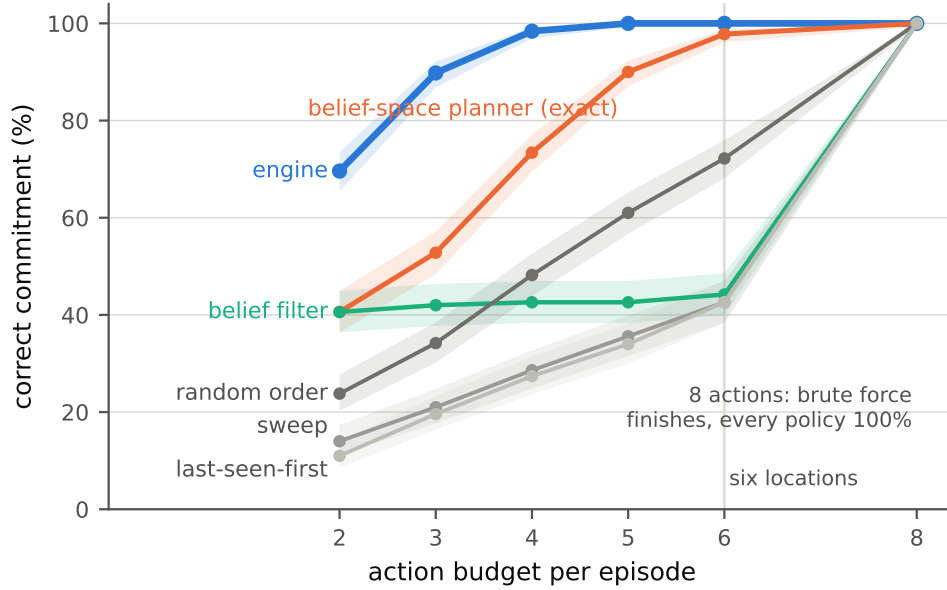


Figure 2: The benchmark below the exhaustive budget. Correct-commitment rate against action budget, 500 episodes per point, bands are Wilson 95% intervals. Brute force finishes at eight actions and every policy reaches 100%; below six, the scripted policies lose most episodes while the engine keeps them. The exact belief-space planner, optimal lookahead over the filter’s own belief, closes about half of the filter’s gap to the engine and none of the rest. The belief filter’s plateau is the cost of argmax-posterior without transport handoff; the random walk’s slope is plain coverage.

seeds. The average moves with the seed set (2.76 on the 25-seed battery artifact, 3.25 on the 12 head-to-head seeds), and this 40-episode run is regenerate-to-verify. Fresh reports are verified before being trusted. In the ledger, conflicting sightings of one object are marked CONTRADICTED by a fixed rule rather than averaged, and multi-anchor hypothesis generation keeps the true carrier chain alive. Deception raises the action count and leaves accuracy untouched.

In these measurements the accuracy gap closes for the two hosted agents. The latency gap and the absence of a replayable trail remain (§5). Those 48 episodes come from one generator and one set of worlds. The next question is whether the same build travels.

4.3 One build, unchanged, in other worlds

One build left the warehouse with nothing changed and was correct wherever it committed: 134/134 on ALFWorld’s unseen split, 95/96 fetches in AI2-THOR, 12/12 scored reports on DARPA SubT worlds, 8/8 on a hospital floor, and every target on a Mars terrain and a seabed. These evaluations grade the transfer of the search policy under the adapter contract of §3. Their discriminating evidence is the planted-tip and staged-decoy rounds, where a wrong commitment is possible and the policies separate.

ALFWorld’s unseen split. The warehouse and its levels are worlds we built, so v0.4 also entered ALFWorld [46], the text-world twin of ALFRED [45]. It has 134 unseen household tasks, a standard 50-step budget, and success judged by the game. The adapter turns template text into claims and engine actions into the game’s admissible-command menu, and hand-codes six task recipes (the fixed clean/heat/cool/examine/place mechanics). Which receptacle to try next, and when to give up, are the engine’s calls. Development used

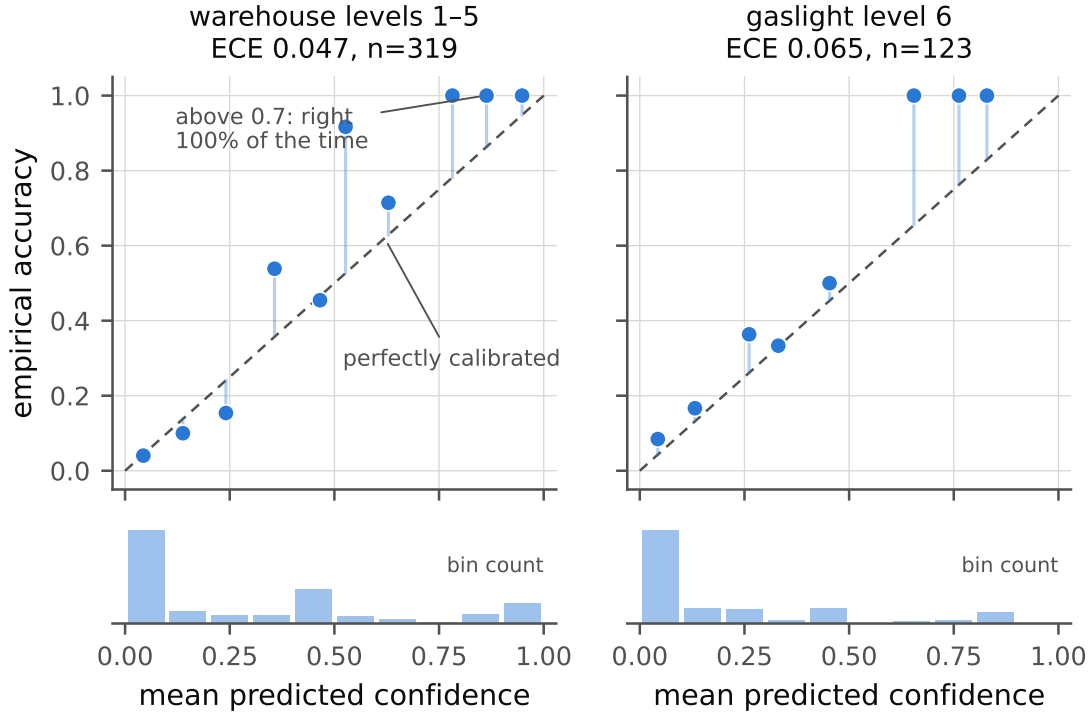


Figure 3: Reliability diagrams with bin counts, from data/artifacts/calibration.json. Dots above the diagonal are underconfident. The engine stays conservative under gaslighting: high confidence is never wrong; the miscalibration that exists sits in the low-confidence bins.

the train/seen splits. The first run scored **129/134** (96.3%) one-shot. For scale, verified against the cited papers, the best single-attempt LLM results are AutoRefine’s 100% (134/134, GPT-5.6-terra backbone with compiled typed artifacts) [35] and AutoManual’s 97.4% (GPT-4-turbo with learned manuals) [8]. Reflexion’s 97% is cumulative over 12 retry trials [44]. ReAct-style agents report 57% on average and 71% best-of-six with a PaLM-540B backbone [55, Table 3], and 79.9% with a current backbone in AutoRefine’s harness [35, Table 1]. No verified system that is LLM-free at runtime exceeds BUTLER’s 37% (best of eight seeds on the unseen split; 46, Table 4, as reproduced in 55, Table 3). BUTLER learns the task mechanics our adapter hand-codes, so that comparison scopes the search policy only.

Scripted ablations show what the benchmark measures. Under identical adapter mechanics a fixed alphabetical sweep scores 124/140 (88.6%) on valid_seen and a seeded random-order walk 134/140 (95.7%), a tie with the engine. Six recipes plus any systematic non-repeating search solve text-ALFWorld, so it cannot separate belief machinery from a random walk. Tracing the seen-split two-object losses exposed an engine defect, a stale class-level sighting that resurrected already-checked rooms (Appendix B gives the mechanism). The fix was developed and validated on the seen split only (134→138/140). The unseen split was then evaluated a second time, a second run of the test set: **134/134** (100.0%). The random-order baseline scores 133/134 (99.3%) unseen. The engine’s edge over a memoryless random walk is therefore +1 and +4 games on the two splits, consistent in direction and bounded by a benchmark at ceiling. The 100% matches the only other verified perfect score, AutoRefine’s 134/134 with a GPT-5.6-terra backbone [35], with no language model running at inference time and milliseconds per decision.

agent	correct (L2–5)	avg. steps [†]	gaslight L6	wall-clock/ep	proofs
robot_mind engine, session 1	48/48	1.75	40/40	~5 ms	100% checked
qwen2.5-7B basic (local), session 1	40/48	3.73	–	~11 s	none
qwen3-8B thinking (local), session 1	45/48	2.07	–	~94 s	none
deepseek-chat (API, strong), session 1	48/48	1.94	–	~9 s	none
robot_mind engine, session 2	48/48	1.83	12/12	~5 ms	100% checked
qwen2.5-7B strong (local), session 2	39/48	3.62	–	~11.7 s	none
gemini-3.7-flash (API, strong), session 2	48/48	1.62	12/12	~10 s	none

Table 7: Head-to-head under the fairness contract (Figure 4). Each LLM row sits beside the engine figure of its own measurement session, on the same 48 episodes. The two session-2 LLM rows were re-measured because their session-1 artifacts were not retained; qwen-strong reproduced its session-1 figures unchanged and gemini reproduced 48/48 with slightly different step and latency figures. The engine’s step count differs between the sessions because the session-2 build carries its learned rule. [†]LLM step averages are over successful episodes only (over all 48 the qwen-basic figure is 4.44); the engine’s coincides because it never fails. Gaslight cells: the engine’s 40/40 is its 40-episode L6 battery; its 12/12 and Gemini’s 12/12 are the 12 head-to-head seeds, where the engine takes 3.25 steps to Gemini’s 2.50. Wilson 95% intervals: 48/48 [0.93, 1.0], 45/48 [0.83, 0.98], 40/48 [0.70, 0.91].

Six simulated environments. Table 8 lists every embodied evaluation with its sensing abstraction, its result and its scope, and Figure 5 plots each row with its Wilson interval.

AI2-THOR find-and-fetch. Twelve unseen iTHOR scenes [25] span all four room types, with placements shuffled per episode and success graded by the simulator’s `PickupObject` signal. Vantages come from an occlusion-tested next-best-view planner [12, 42]. Eight seeds (96 episodes) give **95/96** fetched (99.0%), and all 95 commitments matched ground truth. The one miss, a plate on a side table on seed 314, exhausted the budget with nothing wrongly committed: a perception-coverage residue, not a reasoning error. Precision is 100% by construction, since a commitment fires only on a sighting and oracle visibility flags make every sighting true.

DARPA SubT Virtual artifact search. The `osrf/subt` circuit practice worlds [11] run under DARPA’s scoring rule: a report scores iff the artifact class is right and the position is within 5 m of truth. The engine gets the tile list as bare locations and hunts a seeded target class per episode. Four worlds (two tunnel, one urban, one cave) and three seeds give **12/12** reports scored, in 2–87 of a 250-visit budget. Sensing is section-granular and reads simulator state, so the position half of the rule is inherited from the sensor and the graded quantity is the search. Duplicate-class seeds are deterministic replays (12 episodes, 9 distinct searches), and targets are always present.

Hospital fetch and planted tips. The AWS RoboMaker hospital world has, to our knowledge, no prior published fetch benchmark. Per episode the evaluator relocates one unique-instance item beside a seeded furniture spot, and the engine, told only the class, must find it among 102 real hospital furniture locations. Eight seeds give **8/8** correct at depths of 1–21 visits. The planted-tip variant adds an anonymous REPORTED claim to the ledger naming a spot, true with seeded probability one half. It runs the engine, an evidence-ignoring sorted sweep and a naive tip-follower on twelve seeds, six true and six false tips. All three find every target, 12/12, so visits are the discriminating quantity (Figure 6). With a true tip the engine confirms the reported spot in 1.00 visits, matching the tip-follower, against the sweep’s 6.17. With a false tip it checks the spot once, records the absence and redirects: 14.67 visits to the scripted policies’ 15.50. No policy committed

simulator / task	n	sensing abstraction	result	scope
AI2-THOR fetch	96 ep.	visibility flags + NBV poses	95/96, prec. 95/95	placements not bit-stable
AI2-THOR, real detector	96 ep.	YOLO-World on 1080 ² frames	83/96, prec. 83/83	8 thin/small-object recall, 4 mislabels, 1 sealed in a box; prec. by construction
AI2-THOR, lifelong mind	30 ep.	visibility flags, one persistent mind	95 $\rightarrow$ 10 actions on habit episodes	one house, oracle sensing, habit spot selected
SubT artifact search	12 ep.	section-granular state read	12/12 scored	9 distinct searches
Hospital fetch	8 ep.	6 m live state read	8/8 correct	depth 1–21 varies by draw
Hospital planted tips	12 ep.	6 m live state read	12/12; true tip $\rightarrow$ 1.0 visits	false-tip edge inside noise
Hospital staged decoy	6 ep.	6 m live state read	6/6, 0 fooled	class-match policies fooled 4–5/6
Hospital staged decoy, 30-seed power run	30 ep.	6 m live state read	30/30, 0/30 fooled	pre-registered; class-match policies fooled 16/30 and 26/30
Mars rover terrain (Space ROS demo on Jazzy)	12 ep.	logical 8 m mast sensor, 169 stations	4/4 per arm (engine, sweep, random)	uniform priors: completeness only
Seabed survey (Project DAVE)	12 ep.	sonar range–confidence curve, 70 stations	4/4 per arm	uniform priors: completeness only
Gazebo box hunt	4 ep.	logical camera	4/4 correct	4-bay world
ROS 2 topic demo, diff-drive	4 ep.	logical-camera frustum over bridged poses	4/4 correct	driven over / cmd_vel, no teleports
ROS 2 packaged stack, demo driver	4 + 1 tip ep.	frustum at the odometry pose	4/4 + tip redirected; 20/20 checks	odom drift to 27 m; MoveIt not run
ROS 2 stack on Nav2 + SLAM Toolbox	4 + 1 tip ep.	lidar SLAM localizes; frustum from physics pose	4/4 + tip redirected; 21/21 checks	same sequences; drift ≤ 0.10 m
ROS 2 stack on Nav2 + YOLO-World	4 + 1 tip ep.	RGB camera, YOLO-World, pose-projection association	4/4 + tip redirected; 26/26 checks	finds: same sequences; tip reordered by one nav timeout; 0 wrong bindings
Real pixels: phone camera in a house, 4th sitting	36 + 36 tip ep.	hand-held 1080p clips, YOLO-World per-zone maximum, no association	36/36; tips 18/18 true, 18/18 false	one house, camera carried by hand, zones declared; sweep 24/36, random 27/36

Table 8: Embodied evaluations: one unchanged engine, per-adapter abstractions stated. Locomotion is teleport-abstracted except in the ROS 2 rows, where a diff-drive base is driven with no teleports, and absent in the real-pixels row, where a person carried the camera; none of these numbers is comparable to full-stack embodied leaderboards (ObjectNav, ALFRED vision track, SubT systems track). The plain SubT and hospital protocols grade search completeness and correct commitment only (limitation 10); the planted-tip variant below carries the discriminating embodied evidence.

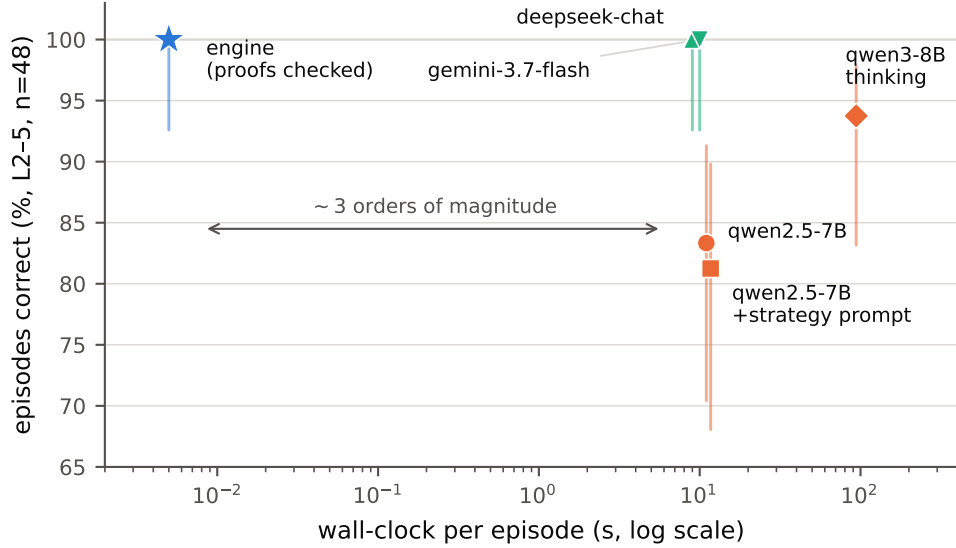


Figure 4: The capability ladder of Table 7: accuracy against per-episode wall-clock, with Wilson 95% intervals. Local 7–8B models trade latency for accuracy and still fall short; the two hosted API models match the engine’s accuracy at $\sim 2,000\times$ its latency. Only the engine’s answers carry machine-checked proofs.

to a reported-but-unverified location anywhere in the campaign. The false-tip counts do not separate the policies (paired sign-flip permutation test over the six false-tip seeds, $p = 0.91$ against the sweep and $p = 1.0$ against the tip-follower).

Staged decoys. The decoy round makes wrong commitment possible. For each false-tip seed a same-class look-alike is staged beside the lied-about spot and the briefing names the specific asset id. A fourth decider joins: a tip-follower that checks instance identity but carries no belief machinery. Over six decoy episodes (`hospital_tips_twin.json`) the class-matching tip-follower commits to the look-alike in five of six and the sweep in four of six. The engine is fooled in none, **6/6** correct at 14.7 visits, its false-tip cost without the decoy. The identity-checking control is never fooled either, at an indistinguishable 15.5 visits, so commitment discipline is what separates safe from fooled. The pre-registered thirty-seed run (`hospital_tips_twin_30.json`) gives the engine **30/30** correct and 0/30 fooled (Wilson 95% [0.00, 0.11]) at 11.7 visits. The identity-checking control is 30/30 correct, and the class-committing sweep and tip-follower are fooled 16/30 and 26/30 times. On the six seeds shared with the first round the engine’s visit count reproduced on four and moved on two (Appendix B gives the counts and the likeliest cause).

Planetary and underwater analogues. The same adapter contract was carried to the Space ROS Mars rover demonstration’s Curiosity terrain (169 stations) and to Project DAVE’s ocean-objects seabed (70 stations). Engine, sweep and random-order arms share executor, budget and sensor. Every arm goes **4/4** on four seeds in both worlds, with zero wrong commitments in 24 episodes (Wilson 95% [0.51, 1.00] per arm). As in the plain SubT and hospital protocols (limitation 10), uniform priors and an always-present target grade completeness and correct commitment only. A random order can beat the engine by luck at $n = 4$, so no efficiency claim is made. The batteries show transfer of the search machinery, unchanged, to planetary and underwater station maps.

Transfer holds under the adapter contract. What the contract still abstracts away is the robot itself, and

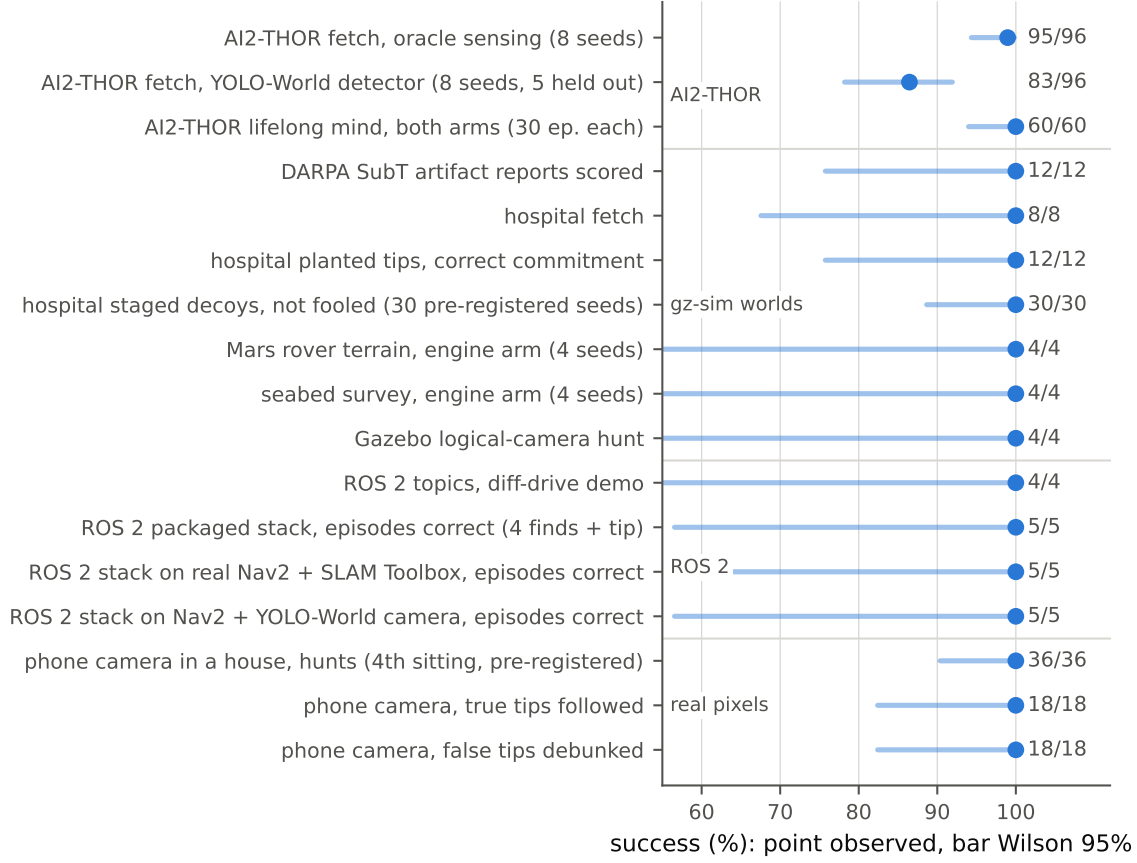


Figure 5: One unchanged engine across every embodied evaluation in this paper. Points are observed success, bars are Wilson 95% intervals; k/n at right. The detector row is the eight-seed battery of §4.4, five of the seeds held out; the ROS 2 rows are the checked episodes of each lane (four staged finds and the false-tip episode), the typed-client check counts (20/20, 21/21, 26/26) being in the text; the last three rows are the pre-registered fourth sitting of the real-pixels battery, a hand-held phone camera in a house.

the next subsection puts real components in its place.

4.4 The robot’s seams

Uncertainties 5 and 6 asked whether commitments derived from a real detector’s raw output stay correct, and whether the decisions survive a navigation stack’s timeouts, aborts and drift. They do, with the engine binary unchanged in every run below. A deployed robot reaches the engine through three seams. Perception turns pixels into typed claims. Navigation turns “inspect here” into motion in a map. Time matters because a mind that runs for weeks is not the mind that runs for one episode. This subsection crosses the first two with a real component in each slot, and §4.5 crosses the third. YOLO-World stood in for AI2-THOR’s visibility oracle. Stock Nav2 with SLAM Toolbox in simulation stood in for the demo driver. A last battery films a house with a hand-carried phone, with no simulator anywhere in sensing.

A real detector in AI2-THOR. The same twelve scenes and all eight seeds ran with the simulator’s visibility flags replaced by YOLO-World [9] on the rendered frame at 1080². The detector is prompted

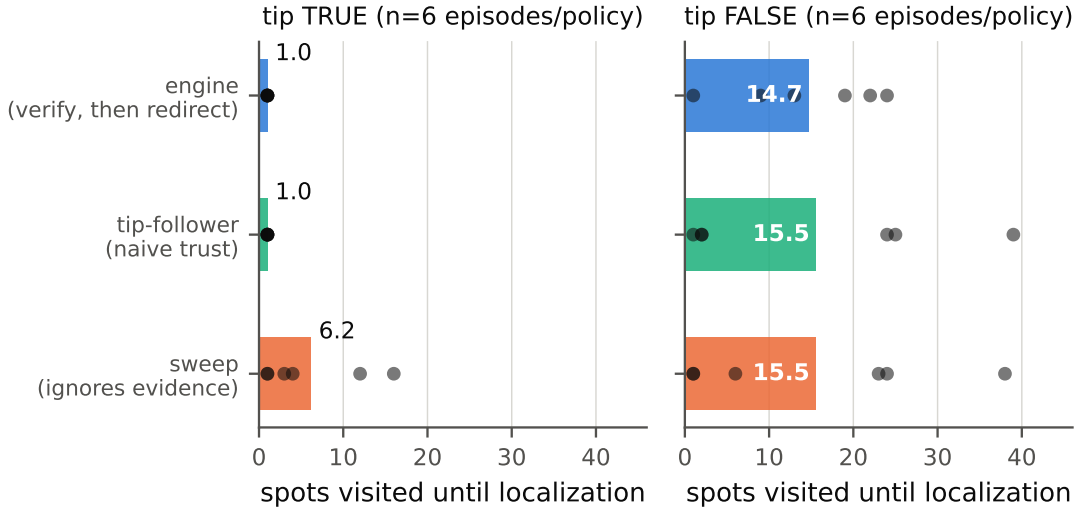


Figure 6: The planted-tip hospital protocol: mean visits per policy with per-episode draws as dots (12 seeds per policy; 6 true, 6 false tips). A true tip collapses the engine’s search to one visit; a false tip costs one verification and a redirect. No policy ever commits wrongly, and only the engine can show *why* the tip was disbelieved.

once with the 62 iTHOR pickupable types and never told the target. A detection is bound to the rendered instance’s segmentation box at $\text{IoU} \geq 0.35$ through a fixed table of eleven visual-synonym groups. It enters the ledger at the detector’s own confidence once it passes a 0.05 claim gate. This association stands in for a tracker. The detector supplies localization and a coarse label, the simulator’s instance segmentation supplies identity, and the association can only accept detector output, never add to it. Seeds 7, 11 and 23 were the development seeds. Seeds 42, 99, 137, 256 and 314 were declared before any ran and launched on the frozen build at commit 8b4a83f, held out from every fix. The engine binary was unchanged throughout. The adapter-side changes (synonym table, claim gate, confirmation ordering) are listed in Appendix B and in docs/EMBODIED.md.

The development seeds fetched 31/36 (86.1%) with 31/31 commitments correct. The held-out seeds fetched **52/60 (86.7%) with 52/52 commitments correct**. All eight seeds together stand at 83/96 (86.5%) with 83/83 commitments correct. The median is 19 actions per fetch and the mean 30 (Table 9, Figure 7). Association is gated by the simulator’s instance boxes, so a wrong-instance commitment cannot occur here. Precision is by construction, and 83/83 checks the seam end to end. The search itself is measured by recall, 83/96 (Wilson 95% [0.78, 0.92]; the held-out seeds alone 52/60, [0.76, 0.93]), and by actions per fetch. Every miss was probed to the detector and reported not-found after the full budget. Eight sit below the detector’s recall from every reachable vantage, four are mislabels localized at IoU 0.84–0.93 but named as something else, and one, a CD sealed inside a box, was never rendered. The 480² first campaign, two start-up crash reruns, one adapter defect, each miss with its detector scores, and the operating point are in Appendix B.

ROS 2, lane one: the packaged node with the demo driver. The engine is packaged as a managed ROS 2 lifecycle node with a FindObject action and a nav2_msgs/NavigateToPose client, so Nav2 replaces the demo driver one for one. A typed certification client (an integration-test suite run against the live stack; the repository’s name for it is kept) passes 20/20 checks. Lifecycle gates hold. Four staged finds succeed at 2, 4, 1 and 3 inspections with proof paths and grasp poses. A false tip injected mid-episode names a bay already inspected and found empty; the engine re-verifies it once, abandons it, and finds the truth (Table 10). The

seed	fetches	commitments correct	median actions	misses, each probed
7 (dev.)	12/12	12/12	15.5	—
11 (dev.)	11/12	11/11	61	fork inside a garbage can
23 (dev.)	8/12	8/8	13	two credit cards lying flat, keychain inside a garbage can, candle on a tank (mislabel)
42	11/12	11/11	20	fork inside a garbage can
99	12/12	12/12	25	—
137	11/12	11/11	19	butter knife inside a garbage can
256	8/12	8/8	17	butter knife on a dining table (mislabel, “Pen”), CD sealed inside a box on a bed, alarm clock on a shelf (never named), plunger on a shelf (mislabel)
314	10/12	10/10	19.5	pepper shaker on a dining table (mislabel, “Soap-Bottle” 0.92), credit card on a chair
held out (5)	52/60	52/52	19.5	
all (8)	83/96	83/83	19	

Table 9: The real-perception battery per seed: twelve scenes each, a target class drawn per seed, YOLO-World on 1080² frames. Seeds 7, 11 and 23 were the development seeds (synonym table, claim gate and confirmation ordering diagnosed on their misses); the five below them were declared before they ran and touched by no fix. Every commitment was correct; every miss was probed to the detector and reported not-found after the 250-action budget. Seeds 42 and 137 each had one scene rerun as a fresh draw after a simulator start-up crash.

demo driver’s stepwise commands let wheel odometry drift to 27 m within the session, so this lane grades arrival and sensing in the odometry frame.

Lane two: stock Nav2 with SLAM Toolbox in simulation. The same stack then ran with navigation served by stock Nav2 in simulation and localization by SLAM Toolbox over a roof lidar (Appendix A inventories the stack). The demo driver was not launched, and the cognition node changed only its goal frame. Result: **21/21 checks**. The four finds and the tip episode reproduced the same inspection sequences the engine chose against the demo driver. Wheel-odometry drift stayed ≤ 0.099 m and SLAM pose error ≤ 0.082 m at every arrival, in one launch attempt at a load average of 0.7 on 16 cores. Each inspection costs 24–38 s of wall time, almost all of it driving at no more than 0.8 m/s. The engine’s decision inside it takes a third of a millisecond (§4.7). A `NavigateToPose` goal that is rejected, aborts, or exceeds the node’s 120 s result timeout returns to the engine as a failed action. It costs one unit of visit budget and records no claim, and the engine re-plans from the unchanged ledger. The tip episode contains one such timeout, on the re-verification leg: five navigation attempts for four inspections, 246 s against 56 s under the demo driver.

Lane three: YOLO-World in the Sense slot under Nav2. The third set of checks keeps the Nav2 stack and swaps the Sense slot. A roof RGB camera (640×480 , 1.4 rad) feeds a `yolo_sense_node` that runs YOLO-World with a fixed generic vocabulary that never names an entity. Each detection is bound to an entity by projecting that entity’s physics pose through the camera intrinsics at $\text{IoU} \geq 0.35$. This pose-projection association is the tracker stand-in, as the instance boxes were in AI2-THOR. The target and three distractors are 0.6 m cardboard cubes with one geometry and one texture. The target stands beside a distractor in three of the four finds and in the tip episode, so identity comes from geometry alone, which is where a wrong-instance binding becomes possible. One adapter-side rule was needed, of the same kind as the AI2-THOR confirmation gate. It adds an inspection-confirmation claim only when the engine’s own verdict resolves the target at the bay the robot stands at. The sensor node and the rule are specified in Appendix B.

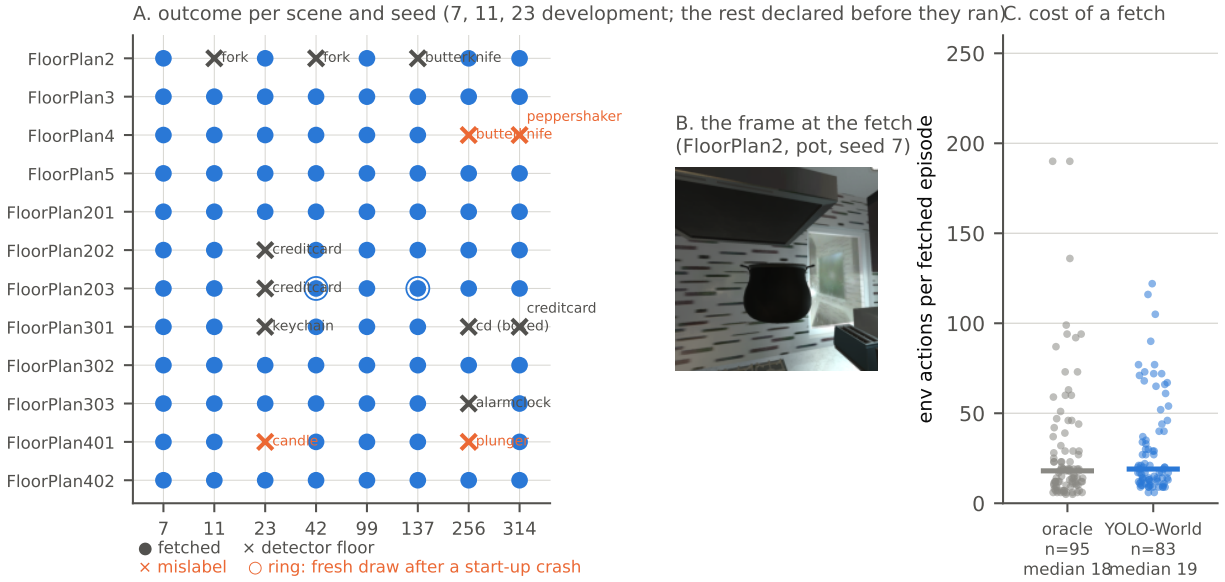


Figure 7: The real-perception battery. (A) Outcome per scene and seed, the target class drawn afresh per seed: filled circles are fetches; crosses are misses labelled with their object, gray for the detector’s recall floor on thin and small objects, orange for a mislabel; the CD marked boxed sat inside a box and was never rendered. (B) The 1080² render as YOLO-World saw it at the moment of a fetch (the detector’s boxes are drawn in the run animations of the artifact bundle). (C) Actions per fetched episode with oracle sensing (eight seeds, 95 fetches) and with the detector (83 fetches), one point per episode, bars at the medians (18 and 19): the detector costs actions on the hard cases, never commitments.

Result: 26/26 checks. The four finds were correct with the same inspection sequences as under the logical sensor. The tip episode verified and abandoned the tip and found the truth. A navigation timeout on its first leg reordered its inspections: bay_3, bay_4, bay_4, bay_2 against bay_4, bay_3, bay_4, bay_2 under the frustum sensor. The decisions were the same, in five attempts for four inspections. The detector named the target at 0.12–0.60 and the look-alikes at 0.32–0.65. Two same-mesh boxes shared the frame at an inspection in 4 of the five episodes, and 11 frames held two or more box detections. Every detection was bound to its own projected entity, with 0 wrong-instance sightings, and each of the 5 confirmations followed the engine’s verdict. Wheel-odometry drift stayed ≤ 0.099 m and SLAM error ≤ 0.048 m. Detector time was 49–403 ms per frame after a 2.0 s first-frame warm-up, in one launch attempt at a load average of 0.86 (ros2_stack_cert_nav2_yolo.json; Figure 8). The timeout’s cause is in Appendix B.

Real pixels: a phone camera in a house. The batteries above still let the simulator tell the adapter which entity a detection belonged to, and this one removes that too. The owner filmed six declared zones of a lived-in house (a sofa, a desk, a kitchen island, a marble table, a blue bin, a navy bench) with a hand-held phone, placing six object classes over six rounds per sitting from a seed committed as a manifest before filming. An inspection decodes one zone’s clip, runs YOLO-World on every tenth frame, and enters the per-zone maximum in the ledger as an OBSERVED sighting at the detector’s own number. There is no instance id, no IoU gate and no association step, so a wrong commitment is possible, and that is what the battery grades. The filming procedure, the recording sheet and the sense model are in docs/PHONE_PROTOCOL.md, and the engine is briefed with the target class and the six zone names, nothing else. The baselines are a zone-by-zone sweep and a seeded random order that commit on the first accepted sighting, and a tip variant reports one

episode	truth	inspection sequence (identical under both)	demo driver (odom frame)		Nav2 1.3 + SLAM Toolbox (map frame)		
			wall (s)	odom drift (m)	wall (s)	odom drift (m)	SLAM err. (m)
0	bay_3	bay_4, bay_3	23.4	0.76	49.9	0.032	0.049
1	bay_1	bay_4, bay_3, bay_2, bay_1	60.1	17.44	147.5	0.099	0.040
2	bay_4	bay_4	18.1	9.58	40.2	0.064	0.043
3	bay_2	bay_4, bay_3, bay_2	31.9	27.03	69.8	0.007	0.028
false tip: bay_4	bay_2	bay_4, bay_3, bay_4 (tip re-verified), bay_2	56.2	8.81	246.1 [†]	0.028	0.082

Table 10: One cognition node checked against two navigation backends (`data/artifacts/embodied/ros2_stack_cert.json`, `ros2_stack_cert_nav2.json`). Every episode is correct with a proof path and a grasp pose under both, and the engine’s inspection sequence is the same under both; under uniform priors that sequence is the fixed sweep bay_4, bay_3, bay_2, bay_1 cut at the truth, so the tip episode is the evidence-driven lane. Wall time is per `FindObject` goal, bring-up excluded. Odometry drift is the wheel-odometry pose against the physics pose at each arrival, cumulative across the session: the demo driver’s stepwise commands slip the sphere-wheeled body, DWB’s 0.5 m/s^2 ramps do not. SLAM error is SLAM Toolbox’s map-frame estimate against physics. Under Nav2 the frustum sensor looks from the physics pose, so neither localization column affects grading. [†]Includes one 120 s `NavigateToPose` result timeout on the re-verification leg, returned to the engine as a failed action (five navigation attempts for four inspections).

zone at 0.75 per episode, half of them true, with a tip-follower as a third arm.

The phone adapter sets a sighting bar of 0.9, the loop’s own confirmation confidence, leaving the engine default unchanged and every native suite byte-identical. Below the bar a sighting is a hypothesis, the search goes on, and an exhausted search commits to its uniquely best-supported zone. The bar and the sense model were chosen on the graded footage of earlier sittings, then pre-registered with the manifest. The first graded sitting scored 12/36, and three fixes preceded the pre-registered fourth. The sitting-by-sitting record is in Appendix B.

The fourth sitting was one run, with nothing changed after seeing it. The engine commits to the right zone in **36/36** hunts (Wilson lower bound 0.90; every class 6/6). It takes 5.03 inspections per hunt, against 2.5 for a sweep that stops at its first accepted sighting. It follows 18/18 true tips and debunks 18/18 false ones, with 0 of 18 commitments at a false tip’s zone. On the same clips the sweep scores 24/36 finds, 11/18 true-tip and 13/18 false-tip hunts correct. The random order scores 27/36 finds. The tip-follower scores 18/18 true-tip and 13/18 false-tip hunts correct. Seventeen of the 36 true sightings were under the bar, the tumbler in every round at 0.83–0.89. Those hunts ended by exhaustion at seven inspections rather than on arrival, and none was decided wrongly.

One was close. In round 19 the phone on the bin read 0.750 and the tumbler on the sofa read “cellphone” at 0.752, so an argmax over the detector would have named the sofa. The engine’s belief after the room was searched stood at 0.51 to 0.49 for the bin, which it returned to and confirmed. That episode’s false tip pointed at the sofa, and the engine still committed to the bin, where the sweep and the tip-follower did not. On identical pixels, and with a commodity detector’s raw output through the same seam, the search discipline and not the detector separates 36/36 from the sweep’s 24/36. This is one house, one camera and one sitting.

Both seams held with a real component in the slot. The third seam is time, and it is what the next subsection measures.

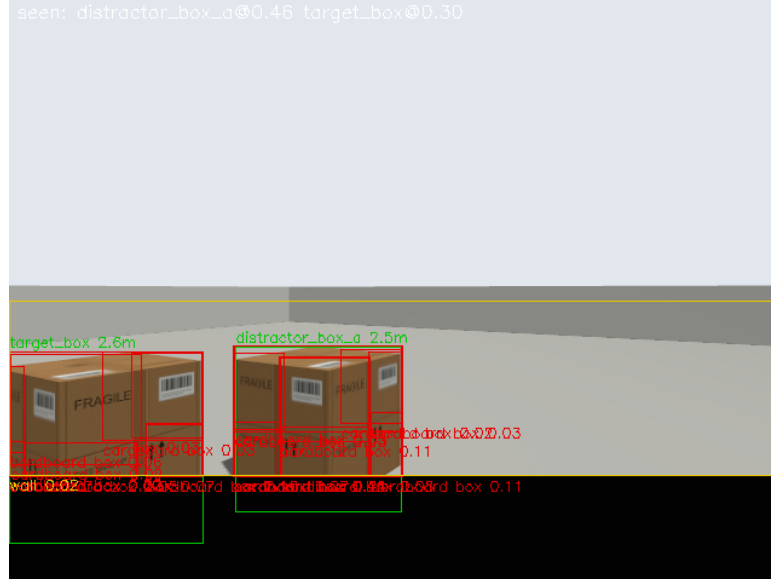


Figure 8: The camera lane, episode 1, the inspection of bay_1 that settled the find: the target box (left, staged 1.0 m beside the bay centre) and distractor_box_a (right) are the same cube and the same texture. Green boxes are the entities’ physics poses projected through the camera intrinsics, red boxes YOLO-World’s detections; the sensor answered target_box at 0.30 and distractor_box_a at 0.46, each bound to its own projected box by IoU, and the engine’s verdict on that sighting confirmed the find. One of the twelve frames the check run keeps.

4.5 A mind that persists

Uncertainty 4 asked whether a memory that never forgets is an asset in a changing world. The asset is a memory that ages beside a search that widens on refutation. With both organs, one mind kept across six days of nightly change holds 12/12 every day; without them, a memory that only accumulates does not.

Transfer and persistence. Two organs separate a mind that cannot enter an unseen room from one that clears a house it has never seen: an exploration drive, then ageing memory. A home world gives find-one 0.96, find-all-five 0.88 and find-collect-brew 0.88 (25 seeds per rung). Procedurally generated houses entered with an empty ledger score 1.00 (W2) and 0.96 (W3) with the v0.3 memory organs, against 0.88 and 0.68 with the exploration drive alone and 0.20 and 0.00 with neither (Appendix B). Re-taking one house daily scores 1.00 with the memory organs, against 0.875 with the drive and fallback organs alone and 0.25 before them.

The dynamic house. The dynamic-house protocol keeps one mind across twelve houses, six days each, with 1–2 items moved nightly. Without the memory organs that mind scored 12/12 on day 1 and 0–4/12 on days 2–6 with drives off, 2–6/12 with drives on (woz_dynamic . json). Per-episode budgets, refutation-driven beam widening and age-decayed evidence restore **12/12 every day**. With realistic 8-hour nights the drive-less mind erodes to 4/12 by day 6, while the patrolling mind holds 10/12.

Habits kept across episodes. Every result in this paper is a cold start. The opt-in lifelong mode (default off, so every number byte-reproduces) persists the robot’s own confirmed fetches and mines habitual-location rules from them through the standard candidate/provisional/validated lifecycle. The rules are advisory

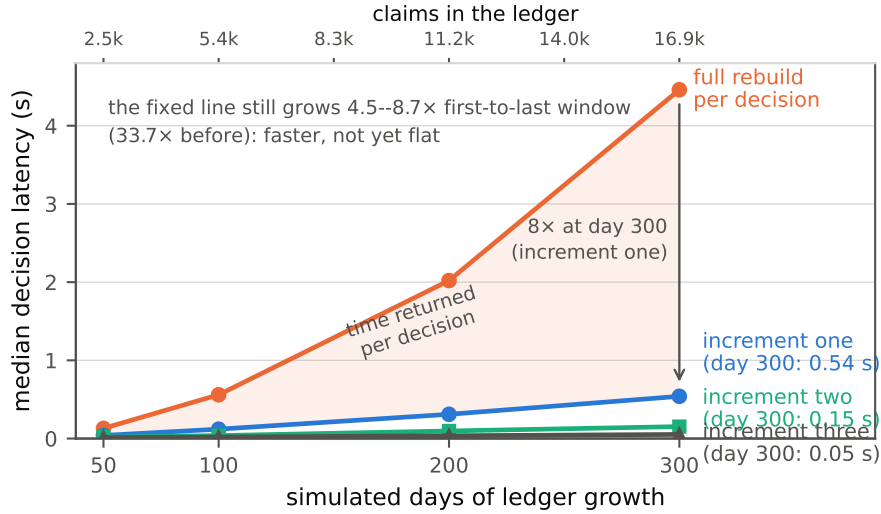


Figure 9: The cost of lifelong memory across three increments of incremental view maintenance (300-day soak; every run byte-identical in its decisions). The full-rebuild and increment-one curves are one session’s paired runs; the increment-two curve is a later session’s run whose own paired before-run reproduced the increment-one curve to within machine-load noise; the increment-three curve is the harness pair of a third session, bracketed by five paired repetitions (`s4_soak_ivm3_reps.json`). The remaining growth has a named cause: the drives’ and meta-controller’s contradicted-claim search and the per-subject sighting walk still walk full claim lists. The upper axis gives the ledger’s claim count at each checkpoint, identical in every run; the marked $8\times$ is increment one’s day-300 ratio ($4.46 \rightarrow 0.54$ s), the band it spans being the time that increment returned per decision.

blind-search priors that an observation always overrides. The habit world is one house with oracle sensing, 30 episodes, and a mug staged on a habitual counter with seeded probability 0.75. A measured sweep chose that counter as the receptacle the cold-start search reaches latest among those it finds within budget (the floor it never finds). So 95 is the worst-case cold cost of a fetchable placement, and the gain is an upper bound. The lifelong mind is indistinguishable from a cold start for twenty episodes. Once the habit validates on held-out fetches (precision 0.80, held-out 0.73), every visit to the habitual counter costs **10 actions instead of 95**. Both arms are 30/30 with zero wrong commitments. A deliberately wrong habit (four episodes off the counter, one on a coffee machine whose parent is another countertop) changes cost by $+1$, -13 , $+7$ and -15 actions. It never costs a commitment. The learning is the engine’s own: no demonstrations, no labels, no evaluator truth. Two protocol facts that bound the reading, and a first curve that validated and saved nothing, are in Appendix B.

The cost of memory. Keeping a mind costs time per decision, and three increments of incremental view maintenance have paid most of it back. Caching the append-only ledger and linearising a contradiction scan came first, for about $4\times$ per simulated day. Increment one then made a 300-day life about $7\times$ faster, increment two a further $4\times$, and increment three a further $2.5\text{--}3.4\times$ over five interleaved paired repetitions. Every run is byte-identical in its decisions (Figure 9). The profiling, the mechanism and the equivalence check behind each increment, with the per-increment figures, are in `docs/STRESS_LATENCY_NOTES.md`.

The maintained views now cost, per decision, $O(\Delta)$ for the belief graph and effective statuses and $O(\Delta \cdot k)$ for contradiction detection. Here Δ counts the claims appended or re-trusted since the last check and k the

claims about one subject. A filtered read matching m claims costs $O(m)$ and the room-visit view over R rooms costs $O(R)$. Two operations still walk the whole ledger of N claims. They are the contradicted-location-claim search in the drive manager and the meta-controller, and the per-subject sighting walk over a history of order N . Per-day cost therefore still grows about linearly with N on this workload, $4.5\text{--}8.7\times$ for $6.6\times$ in claims ($2.5\text{k} \rightarrow 16.9\text{k}$). The per-checkpoint chain across the four sessions' curves in Figure 9 reads $0.13 \rightarrow 0.0125$ s/day at day 50 and $4.46 \rightarrow 0.053$ s/day at day 300. It is cross-session, so indicative rather than paired.

Every battery to this point places the target somewhere in the world. The next one does not.

4.6 The absent target

Uncertainty 8 asked whether the commitment discipline holds when the object is not in the building at all. The discipline held in every battery above, and every one of them put the target somewhere in the world. This battery takes it away deliberately, and locates where the discipline ends. The engine has two ways to commit. The direct path commits on a fresh, uncontradicted room sighting whose confidence reaches the resolve bar. The exhaustion path opens only when that bar is above zero. It fires once no sighting reaches the bar and every room is either named by a hypothesis or already observed (sighted or found empty). It then commits to the uniquely best-supported candidate rather than return no verdict.

Controls, target present. With the target in the world the engine finds it in 300/300 episodes at each bar, with and without the deployed learned rule. Shrinking the action budget breaks that figure ($120/120 \rightarrow 74/120 \rightarrow 53/120 \rightarrow 23/120$ at budgets 3, 2, 1 on 40 seeds), so it is falsifiable. But it measures search success, not commitment correctness: the control confirms after 3.5 actions on average and never spends its budget. A length-matched control therefore suppresses confirmation, spends all eight actions with the object present, and scores the final verdict against live world truth. At bar 0.9 it holds a verdict in 196/300 episodes, all from the direct path, and 0/300 name a wrong room. The other 104 end with no verdict: their sighting aged past the direct path's freshness window, and the exhaustion path stays gated off once a sighting has cleared the bar.

The boundary, located. At bar 0.9, with the target nowhere and the stale in-building reports left in place, the exhaustion rule names a place for the missing object in 121/300 episodes (Table 11). The split is 43, 43 and 35 at levels 3, 4, 5, all exhaustion verdicts naming rooms rather than containers. Belief on the phantom is $0.226 / 0.509 / 0.743$ (min / median / max). The count is 126/300 on a disjoint seed range and 121/300 with the learned rule removed. The rule fires 799 times across the arm's 3,000 reasoning calls; all but the 121 are retracted by the inspection they trigger, which finds the room empty. World truth reads *unknown* in 300/300 episodes and the seed streams are byte-identical to the controls (Appendix B), so absent and present worlds are the same worlds.

The same rule, seen from the other side. The exhaustion rule also fires with the object present, whenever the beam's candidate rooms are refuted before the object is seen. At bar 0.9 it fires transiently in 74/300 control episodes and names a room that is wrong at that moment. The inspection it triggers retracts it, and every such episode still ends on the right room. Under a three-action budget there is no time for the retraction: the rule decides 24/120 control episodes and is wrong in 19 of them. So the boundary is not absence alone. It is the exhaustion rule at a raised bar whenever the candidates run out before a sighting clears it, and absence is the condition under which that always happens. The one published run of the rule at a raised bar with the target present is the fourth real-pixels sitting (§4.4). There, 17 of the 36 hunts ended by exhaustion after every zone had been inspected, all on the right zone. That budget, twelve actions over six zones, always allows a re-inspection. No published number runs the rule with an absent target or with a budget too small to re-inspect.

arm	held verdict at the end	wrong at the end	no verdict at the end	transient wrong verdicts, retracted
control: target present, confirmation on	300 (direct path, all confirmed)	0	0	74 episodes (exhaustion path)
length-matched control: confirmation suppressed	196 (direct path)	0	104	74 episodes, the same ones as the row above: the transient precedes the sighting, after which the exhaustion path is gated off
absent: stale reports left in place	121 (exhaustion path)	121	179	799 exhaustion firings across 3,000 reasoning calls, 678 of them retracted by the inspection they triggered
control at a three-action budget (40 seeds, 120 episodes)	24 exhaustion decisions	19	—	—

Table 11: Verdict accounting at bar 0.9 (`absent_target.json`, 300 episodes per arm, 120 in the last row), in the rows of the Table 2 vocabulary.

What the measurement specifies. The mechanism is a missing capability, not a defect: when the candidates are exhausted the engine commits to the best-supported one because the type system has no vocabulary to decline. A justified not-found verdict is the top item of future work, and this battery is its acceptance test (limitation 13). The 121/300 was re-derived by a classifier that does not use the harness’s reporting code (Appendix B).

Two arms of the sweep are diagnostics rather than results (Appendix B). At bar 0.0 the absent arm shows 36/300 transient direct-path verdicts and 0/300 held at the end. They rest on a level-5 camera artefact (0.45–0.62) that passes the freshness test while the confidence test is vacuous at zero, so the 0/300 is an anchor-freshness window, not restraint. The evidence-stripped arm returns the same uniform belief vector in 300/300 episodes, a wiring check that no verdict is invented from nothing. Worlds are synthetic warehouses; only the resolve bar is shared with the real-pixels battery.

The boundary is a missing verdict, not a broken one. What the engine does under attack, and what it costs, are the last measurements.

4.7 Robustness and cost

Uncertainty 3 asked whether evidence hygiene survives an adversary who plants confident false claims, and the engine’s price per decision had to be measured beside it. Nine suites built to break the engine leave all 1,860 unseen episodes correct and never confidently wrong under 320 planted lies per episode, and one decision costs a third of a millisecond on a laptop core.

Stress battery. The nine suites attack different surfaces. Seed fuzzing ran 1,860 episodes (310 unseen and degenerate seeds $\times$ 6 levels) and property fuzzing threw 3,400 random claim/belief soups at the ledger. Other suites fed it malformed and hostile input, ran a 300-day dynamic-house soak and checked determinism across parallel processes under varied hash seeds. Gaslighting planted up to 320 lies per episode, and crash recovery hard-killed the engine after 73 committed claims, all durable. Resource limits scored a wide belief beam in 0.07 s, validated a 3,000-deep proof chain in 0.28 s and queried a 40,000-claim ledger in 0.38 s. The ninth is an unscreened self-repair sabotage sweep (120 random corruptions). The artifact JSONs ship with the kit; the full protocol notes stay with the engine.

All 1,860 unseen episodes were correct, each under 32 ms in the worst case, and byte-identical across four processes. Under floods of up to 40 waves of eight planted claims (320 lies per episode) the engine never reported success at a fabricated location. It was 100% correct through 160 lies per episode and 93% at 320,

not-found in the rest. The ledger survived hard kills and concurrent access. Across the 120 config sabotages the self-repair loop never crashed, never adopted a regression and never adopted its placebo.

The battery found five defect classes (one at two code sites), all of one kind: correct reasoning, ungraceful failure on hostile or corrupt input. An unbounded timestamp crashed the ledger, a corrupt saved-config file crashed its loader, and a bad command-line level printed a raw stack trace. A tampered config value outside $[0, 1]$ pushed a computed claim confidence out of range at two claim-construction sites. Each fix refuses or clamps at the typed boundary: the suite of that day (133 tests) green, replays byte-identical, ALFWorld re-run from scratch at 134/134. A sixth reported failure was a defect in the test itself, since corrected. Per-suite figures, the defects and the sabotage sweep are in docs/STRESS_LATENCY_NOTES.md.

Latency. Table 12 collects this paper’s latency figures, each from a different instrument, and rows are not comparable without their basis.

quantity	value	basis
one decision, fresh ledger	0.33–0.34 ms median; p95 0.70–0.77 ms; p99 < 1.1 ms	1,025 decisions over 360 held-out episodes, one pinned laptop core, near-idle machine
one episode, fresh ledger	2.8–3.1 ms median; 4.8–5.4 ms p95	the same run
one episode, head-to-head campaign	~5 ms wall-clock	Table 7’s session, unpinned, beside the LLM harness
one episode, hosted LLM agent	~9–12 s	end to end through the provider’s API, network, queueing and tool loop included; local 7–8B models 11–94 s
one simulated day at day 300 of the soak, after increment three	0.040–0.081 s (0.157–0.248 s before)	five interleaved paired repetitions, 33–59% background load
first-to-last-window growth of that cost	4.5–8.7× after (10.5–15.9× before), against 6.6× in claims	the same five repetitions; the least stable number in the battery

Table 12: The latency quantities of this paper on one page, each with the instrument that produced it.

On the 360 held-out warehouse episodes (levels 1–6, seeds 10000–10059; 1,025 decisions) one decision on a fresh ledger takes 0.33–0.34 ms median on one pinned laptop core. The p95 is 0.70–0.77 ms, the p99 under 1.1 ms, and the adversarial level costs 0.40–0.47 ms (Figure 10). Peak resident memory is 53–60 MiB, the runtime dependency closure 14 MiB and the wheel 170 KiB. These are laptop-CPU figures. No embedded or ARM board was measured (docs/EDGE.md), and the profile’s run details are in docs/STRESS_LATENCY_NOTES.md.

Self-repair. v0.4 closes a metacognitive loop with no language model and no human inside it. Each organ’s implicit predictions become settled expectation records, and systematic gaps become typed violations that name the implicated constants. Each violation scopes a finite menu of one-edit repairs, which race on the failing scenarios and are adopted only if every guard suite holds and held-out validation does not regress. Each cycle also races one placebo, a mutation of a non-implicated field, which is never adopted and always recorded. Its one departure from the Darwin Gödel Machine is a typed deterministic repair vocabulary in place of that system’s LLM mutator [58], and the exam is fault injection with sealed grading. Two sealed fault-injection exams recover every detectable single-field sabotage at 98–100% with zero false repairs and zero placebo adoptions; one no-archive lane fails by construction; the exams are in Appendix B. Limitation 9 states the four ceilings the loop does not cross.

Failure modes, mitigations, status. Table 13 gathers the failure modes met in this paper with the mitigation and measured status of each.

§5 reads these results together.

failure mode	mitigation in the engine	status
hostile or corrupt input (malformed claims, oversized fields, corrupt saved config)	refuse or clamp at the typed boundary	five classes found by the battery, fixed, behaviour-preserving
planted false sightings at scale	verify before trust; contradiction and absence bookkeeping	100% correct through 160 lies/episode, 93% at 320; never confidently wrong
very old evidence loses its recency ordering	an ordering-preserving treatment, default off	measured; 0 of 360 re-ranks (limitation 4)
detector recall floor (thin, tiny objects)	a missing detection cannot create a sighting; not-found after budget	8 (plus one target sealed inside a box, never rendered) of 96 episodes; 0 wrong commitments
detector mislabel (a jar candle read as a cup)	synonym groups, added only after two independent spawns	4 of 96, reported not-found; a Candle/Cup synonym enters on a second independent spawn
a learned habit that is wrong	rules advisory only, held-out validation before deployment	−15 to +7 actions, 0 wrong commitments ($n = 4$)
latency growth with lifetime memory	incremental view maintenance	$\sim 7\times$, then $4\times$, then $\sim 3\times$; 4.5–8.7 $\times$ growth remains (limitation 11)
a sabotaged configuration constant	expectation monitor, guarded one-edit repair, placebo control	recovers pre-screened single-field faults; one lane fails by construction (limitation 9)
sensing from drifted wheel odometry under a map-frame navigator	vantage from the localized pose (physics, in sim); drift and SLAM error recorded per arrival	Nav2 run 1 wrong at 6.7 m drift; 21/21 run drift ≤ 0.099 m
SLAM starving the navigator (scan queue overflow, stale map→odom)	configuration: sparser graph, longer transform tolerance	Nav2 run 4 answered not-found, no wrong commitment; run 5 21/21
a navigation goal that aborts or times out	failed action at the executor seam: one unit of budget consumed, no claim recorded, re-plan from the unchanged ledger	one 120 s timeout in the 21/21 run's tip episode (5 attempts, 4 inspections); run 4: 8 attempts, not-found, no wrong commitment
wrong-instance binding under a real tracker	re-inspection and absence bookkeeping (the verify loop)	exercised in the camera lane: 4 episodes with two same-mesh boxes in view, every detection bound to its own projected entity, 0 wrong-instance sightings, 0 wrong commitments; the verify loop stays the mitigation for a tracker's binding error (limitation 12)

Table 13: Failure modes met in this paper, the engine's mitigation, and the measured status of each. Every row with a number points at a shipped artifact.

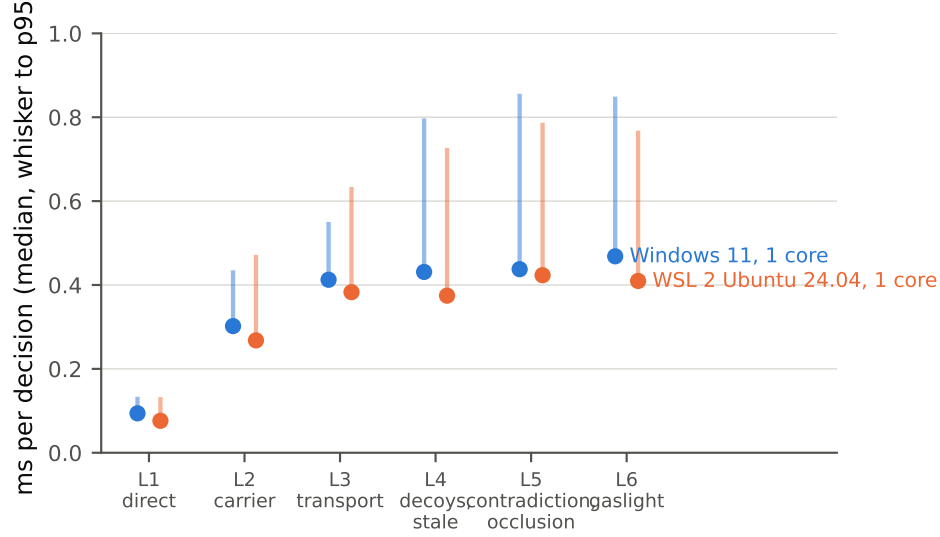


Figure 10: Per-decision latency by benchmark level on one pinned laptop core (Intel i9-11900H): median as the point, whisker to the p95, Windows 11 and WSL 2 Ubuntu 24.04. The adversarial level costs under half a millisecond; the two platforms agree to within a few percent.

5 Discussion

The results establish three things. A deterministic decision layer reaches accuracy parity with two hosted LLM agents on identical evidence. It adds four properties neither of them supplied here: about $2,000\times$ lower per-episode latency (5 ms locally against 9–12 s end to end through the API), byte-identical replay, no per-call model fee, and a machine-checkable derivation record behind every answer. Below the exhaustive budget the machinery is necessary as well as efficient, because an exact belief-space planner over the same evidence closes about half of the plain filter’s gap to the engine and none of the rest. And one build crossed the perception and navigation seams with real components in the slot, and stayed correct wherever it committed.

What the LLM comparison means is bounded by what it measured. On these 48 episodes no accuracy gap is detectable for the two hosted agents, so what separates them from the engine is structural rather than a difference in reasoning quality: latency, cost per call, offline operation and the trail. Against trained robotics stacks the comparison is one of requirements rather than benchmarks. The one learned component inside the decision loop is a drop-in detector on the perception seam (§4.4). The classical belief-space line is raced directly as an exact planner, and the open-vocabulary line supplies front-ends for that detector slot rather than rivals at this layer.

The deployment properties follow from the design rather than from any one benchmark. A deployment is briefed rather than trained: typed goals over a closed ontology, no demonstrations, no data collection and no fleet. Operation is offline and private, every run replays byte-identically, and every decision is contestable afterwards from its ledger. The engine fits inside a control loop at a third of a millisecond per fresh-ledger decision on one laptop core (§4.7). That holds for a fresh ledger per task or for consolidation between tasks; an unbounded lifelong ledger’s cost grows until the remaining view-maintenance increments land (limitation 11). Hospital logistics and SubT-style search, the two settings modelled here, need exactly those properties.

Replayable decisions, offline operation and determinism are evidence an assurance case or a post-incident audit can use. Test, evaluation, verification and validation (TEVV) processes and safety standards such as ISO 13482 [22] ask for evidence of that kind. This paper claims conformity with none of them. The Machinery Regulation (EU) 2023/1230, applicable from January 2027, reserves mandatory third-party assessment

of safety software for systems “with a fully or partially self-evolving behaviour using machine learning approaches” and exempts “software incapable of learning or evolving” (Recital 55). The decision layer as shipped, with every learned element off by default, is of the second kind, which is an observation about the text rather than legal advice, and it holds only while learning stays outside the safety function.

6 Limitations and threats to validity

Read as threats to validity, items 1, 2, 4, 5 and 10 bound *external* validity, and items 3 and 8 and the detector run’s by-construction precision bound *construct* validity. Items 6, 9 and 11 are measured ceilings, item 12 bounds the evidence, and item 13 is a measured missing capability. Items 7 and 14 are disclosures, of provenance and of the audits’ independence.

1. **Home-field advantage.** The engine’s rules were designed for the world distribution the generator samples; the LLM sees it cold. The result shows how grounded competence is built for a domain, not domain-general superiority.
2. **Bounded world, one task family.** Six locations, LOCATE goals, a closed ontology. The larger embodied worlds (102 hospital furniture spots, the SubT tile lists) run under the same ontology and the same goal type (§4.3).
3. **The LLM comparison is bounded by our scaffolds.** The two hosted LLM agents already match the engine’s accuracy here, and stronger agentic scaffolds could lower their step counts further. The harness accepts any Ollama/OpenAI-compatible endpoint and publishes per-episode data, so the comparison is open to extension.
4. **Hand-set constants remain.** Priors, reward weights, the evidence half-life and the staleness horizon live in one versioned config. The half-life was chosen a priori, not fitted to the dynamic-house results it was tested on. An autonomous optimizer with mutation power over every numeric constant found no improving mutation and rolled itself back (`data/artifacts/evolved_config.json`). The hand-set defaults are therefore also the optimizer-validated ones, a statement about the search, not about optimality. The current rule set, v0.2.0, followed temporal probes that exposed two defects after the first campaign (Appendix B). Original-campaign and post-fix figures are both reported (1.75 and 1.80 actions on L2–5). One belief-arithmetic choice is measured rather than settled: how evidence that has aged all the way down is ordered, where the shipped treatment loses the recency ordering between two very old sightings. A design study ran three treatments on the full benchmark: an ordering-preserving variant re-ranks the top hypothesis in 0 of 360 episodes and moves ECE by -0.008 to $+0.007$ across the four calibration cells, while a decay-to-prior variant re-ranks 35 of 360 and degrades shipped-protocol calibration by 0.038 (the ECE cells and the re-rank counts are from the study’s artifact and log, which stay with the engine). Both exist as a default-off engine option; every number in this paper is from the shipped treatment and byte-reproduces with the option in place.
5. **The dynamic house is one change model.** Items move uniformly at random; organs that would learn movement structure are untested.
6. **Consolidation is measured neutral here.** Nightly consolidation compresses repeated-observation runs into summary claims, with the originals superseded, never deleted. It is behaviour-neutral in the dynamic house: `woz_dynamic_reflect.json` equals `woz_dynamic_full_decay.json` in every daily outcome and action count, because passive perception already deduplicates re-sightings. Its benefit under denser sensor streams is expected, not claimed.

7. **Development disclosure: an LLM built this system.** The engine contains no language model at runtime, and every benchmarked decision is deterministic symbolic computation. The system itself was designed, implemented and debugged by an LLM assistant (Anthropic’s Claude) under the author’s direction. Claims of the form “no LLM anywhere” are claims about the *artifact*, never about its development process.
8. **No error bars beyond binomial intervals.** Every table reports raw counts from single deterministic runs; Wilson 95% intervals for the key proportions are in the captions and in Appendix B. The decoy protocol is directional, not powered: 0 of 6 fooled has a Wilson upper bound of 0.39. The pre-registered 30-seed run gives 0/30 fooled at 11.7 visits, upper bound 0.11 (§4.3; launch log in Appendix B). Seed-family variance is unquantified and single-machine, with one visible instance in Appendix B, and one-episode deltas are labelled as inside binomial noise, in the LLM comparison of §4.2 too.
9. **Self-repair has four ceilings no published system removes.** The loop can only fix defects expressible in its mutation space and visible to its expectation monitor. It cannot invent a missing representation, fix a wrong objective, repair its own diagnostic machinery, or see failures it holds no expectations about.
10. **The embodied evaluations grade search, not robotics.** Locomotion is teleport-abstracted except under the ROS 2 stack, and sensing reads simulator metadata or physics state except in the real-detector battery, whose instance association is a tracker stand-in (Appendix B). Under uniform priors the plain SubT and hospital protocols grade search completeness and correct commitment only. The planted-tip variant (§4.3) discriminates on true-tip exploitation, and its staged-decoy round measures commitment discipline directly: engine 0/6 fooled, class-matching policies fooled in 4–5 of 6. A full-stack robot could fail where these adapters succeed. Mapping, control and vision are outside what these evaluations measure.
11. **Latency grows with lifelong memory.** Three increments of incremental view maintenance have cut a 300-day soak $\sim 7\times$, then $4\times$, then $2.5\text{--}3.4\times$, with byte-identical results (docs/STRESS_LATENCY_NOTES.md). Latency still grows $4.5\text{--}8.7\times$ from the first window to the last, against $6.6\times$ in claims. Constant latency over an unbounded lifetime needs the remaining increments, an ordered contradicted-claim index and an event-time-sorted per-subject sighting index. Correctness is unaffected throughout.
12. **Scope of evidence: the decision layer, in simulation.** No physical robot has been run: the work was conducted without lab hardware, and the evaluations isolate evidence-grounded search and decision auditing from locomotion, mapping, manipulation and perception (Appendix B). The two real components ran separately and together (26/26 checks); the claim is that the engine survives each substitution and their combination, not that a tracker’s binding errors were measured. The real-pixels battery’s 36 hunts are one house, one camera and one sitting (Wilson lower bound 0.90), and one was decided by a belief margin of 0.02. Physical deployment follows from the ROS 2 blueprint of Appendix A, and the repository, seeds and artifacts let an independent group reproduce every number before that step (see the reproducibility statement).
13. **A justified not-found verdict is specified but not yet built.** The engine’s verdict vocabulary covers where a thing *is*; the top item of future work is “it is not here, and here is the proof I looked properly.” GoalStatus carries no refusal state, and a null verdict spans “no verdict” and “budget exhausted mid-search”, so no commitment in §4.6 is a silence, not a judgement. That section measures the boundary at the raised bar of 0.9: 121/300 phantom verdicts for an absent object, against 0/300 wrong among the length-matched control’s 196/300 held verdicts. With the object present the rule fires

transiently in 74/300 episodes and decides wrongly in 19/120 under a three-action budget, so the boundary is not absence alone (Appendix B). What remains is a named addition to the type system and its proof obligation, exhaustiveness over the rooms rather than the beam, and its acceptance test exists.

14. **Every audit was a separate session of the same assistant.** The engine, the benchmarks, the adapters, the red-team reviews, the pre-submission audit and this paper share one author and one LLM assistant; where the paper calls a review “separate” or “outside” it means a fresh session of that assistant with its own charter, never an outside group. The artifacts and the kit are shipped so that this can change, and no outside group has yet regenerated the kit’s numbers; §8 makes that regeneration a gate on every objective.

7 Related work

The neighbouring literatures supply every part this decision layer is built from, and none of them supplies the certificate. This section places each beside the work: object search first, then the certificate and provenance lines, then agent memory and the embodied benchmarks.

Object search with a certificate. We found no object-search or object-goal navigation system that attaches a replayable derivation record, a completeness certificate or a justified refusal to its search commitments. In the systems we examined, a published “found” or “not found” is a threshold on a detector score or an embedding similarity. Machine-checkable proofs exist elsewhere in planning and are the parents of the trail here (the paragraph on certificates and provenance below), so the claim concerns search commitments only.

The open-vocabulary line is where perception and exploration are strongest. VLFM builds language-grounded frontier maps [56], OK-Robot does open-knowledge pick-and-drop [29], and WildOS searches unstructured outdoor terrain onboard [43]. SCOUT distils relational knowledge into lightweight scorers, keeping the language model out of inference, and ships the SymSearch benchmark [32]. R2F scores ray frontiers with embeddings [4]. These are front-ends for the detector slot of §4.4, not rivals at the decision layer. OSG Navigator casts the task as a POMDP over open scene graphs [30]. The confidence-calibrated evidence arbitration of Zhao et al. [59] is nearest in spirit to evidence-driven decision-making, without a ledger or a proof. The classical belief-space line is the baseline family for the external head-to-head of §8. It comprises pomdp_py with its multi-object search domain [60], multi-resolution search in 3D [61], and correlational search with a real detector in AI2-THOR [62].

The nearest published guarantees attach elsewhere. KnowNo gives an LLM planner a conformal abstention with a statistical bound on task completion [41]. Conformalised semantic maps give a user-set mission-completion rate for reaching a *known* goal while avoiding semantic hazards [48]. Neither certifies where an unknown object is or that it is absent. Both are templates for the statistical layer of O1 in §8. TRACE states the auditability premise this engine is built on, that investigators must be able to reconstruct why a robot decided, scored by traceability metrics [50]. Here the trail is a proof path a checker replays, and the decisions it explains are graded against hidden truth.

Certificates, provenance and refusal in adjacent fields. Proof-carrying plans give a resource logic in which a plan is a function whose pre- and post-conditions are types, formalised in Agda so planner output becomes proofs [21]. Unsolvability certificates certify that *no* plan exists [17]. They are the nearest published analogue of the NOT-FOUND certificate O1 specifies, and O1 inherits their proof obligation: a checkable argument that the search space was covered. Luckcuck et al. [31] survey formal specification and verification of autonomous robots. That line certifies properties of the controller or the plan, where the trail records the derivation of one commitment, so the two are complementary.

The trail descends from data provenance. Why-provenance [7] and provenance semirings [20] are the formal parents of “which inputs, by which rules”. ECHO brings a bi-temporal, revision-typed memory with checkable provenance to language agents [34]. None of these selects a robot’s next action or is graded against hidden truth. Refusal has a statistical home in classification with a reject option [10] and selective prediction [18]. Both trade coverage for a bounded error rate, the construction O1’s conformal layer and KnowNo share. Epistemic planning [6] reasons formally about what an agent knows and does not know, the closest formalism to a justified “not found”. Tru-POMDP builds particle beliefs from LLM-generated hypothesis trees and plans over an open-ended POMDP [49], so hypothesis structure inside belief-space planning is not new. It lacks the evidence ledger, the typed statuses and the replayable record. LLM+P hands language-described planning problems to a classical planner [28], the LLM-to-formal line beside which the hybrid mode of §4.2 belongs.

Agent memory systems and cognitive architectures. Cognitive architectures [3, 26] pioneered mind-from-modules over production rules. The engine ships a small fixed rule set plus induced rules that pass a validation lifecycle, and no production system. Truth-maintenance systems [14, 16] keep justifications for beliefs. The ledger’s status and supersede machinery descends from them, and proof paths are justification records with time constraints and replay added. Multiple-hypothesis tracking [40] keeps competing tracks alive. Multi-anchor hypotheses are that idea over evidence rather than sensor returns. Belief-space planning [23, 53] chooses actions under state uncertainty with a model and lookahead, without evidence ledgers, proof paths or rule induction. It is the nearest principled competitor, where we choose by information gain over the belief the ledger defines. §4.1 races it under the identical contract as an exact finite-horizon planner over the filter’s belief and the true sensor model, the optimal policy at this size. It closes about half of the filter’s gap to the engine and none of the rest.

LLM-agent memory systems give a language model a store to consult: text, or a graph of facts, sometimes with time. MemGPT [33], Zep’s temporal knowledge graph [39], and reflection and manual-building agents [8, 35, 44] are the examples. Zep’s data model is the closest to the ledger (bi-temporal facts, provenance, supersede rather than delete), but it stores memory rather than closing a hypothesize–act–verify–learn loop. In all of them a model that reasons in language consults the memory and makes the decision, and nothing certifies the path from memory entry to action. They handle open-ended language this engine cannot. Belief-graph agents for text games [1] learn the state tracking the ledger performs symbolically. LLM-grounded task planning [2, 38, 47] attacks the same embodied problems from the opposite direction, a language model planning over grounded affordances or scene graphs. It is the natural opponent for a future embodied fairness contract. Neuro-symbolic agent research argues for the hybrid this system instantiates in closed-domain, auditable form. The self-repair loop draws on expectation-violation metacognition [13], MAPE-K [24], the Darwin Gödel Machine [58], Recuris [57], PAST-Bench [54] and, more distantly, generate-and-validate program repair [27].

What is ours is the combination and its guarantees. The ledger is the only world model. Statuses are typed, so a hypothesis cannot become an observation by scoring high. Every action’s inputs are replayable, the run is deterministic from one seed, and no language model runs at decision time. The trade is a closed ontology, structured input only, and bounded worlds.

Embodied evaluation. AI2-THOR [25] and ALFRED [45] supply the interactive scenes. Object-goal navigation [5] and semantic object search study the same task with learned perception and locomotion, which the evaluations here abstract away to isolate the search policy. The next-best-view planner is the classic occlusion-tested set-cover formulation [12, 42]. The DARPA Subterranean Challenge [11] contributes the worlds and the scoring rule. Full SubT systems [51] solved SLAM, locomotion and communication jointly, none of which is claimed here. We found no prior published fetch benchmark on the AWS RoboMaker

hospital world. The contribution is the working synthesis plus a benchmark methodology. That methodology is hidden-truth grading with a fairness contract for LLM comparison, a sealed fault-injection exam for self-repair, and embodied evaluations, with stated abstractions, in which only the engine varies. To our knowledge these have not been published together.

8 Proposed research and development

The results establish the first half of the innovation of §1.1: a commitment, “the object is in room X ”, that carries a machine-checkable derivation record, at accuracy parity with two hosted LLM agents and with the best published ALFWorld score. The second half exists nowhere yet. It is a refusal, “it is not here, and here is the proof I looked properly”, with a certificate of the same kind, and it is the next objective. Two boundaries were measured rather than resolved. §4.6 located the first, where an exhausted search names a room instead of declining, and it becomes O1. §6 states the second, a per-decision cost that still grows with lifetime memory, and it becomes O2. The remaining objectives carry the work to a real tracker, to a physical robot and to external benchmarks. Each objective names its challenge, its management, and a falsifiable acceptance test on a committed artifact. The method is the one every result here used: hypothesis, mechanism, hidden-truth grading (pre-registered where the world is not seeded), adversarial review, fix, then byte-for-byte re-verification of every earlier battery before the fix is kept.

O1. Certified refusal. *Task:* extend the verdict type from {found at X , no verdict} to {FOUND(X , proof), NOT-FOUND(coverage certificate), UNDECIDED(budget)}, with a refusal state in GoalStatus. Exhaustiveness is defined over the location registry, not the hypothesis beam: only the robot’s own OBSERVED absences within a freshness window count as coverage, never reported tips. The certificate is a proof object (absence claims, their times, the declared sensor recall, the movement-event window) that the kit’s checker validates from the ledger export with no engine code. A commitment requires a valid coverage claim on every other room and a belief margin above a threshold, else UNDECIDED, replacing the exhaustion argmax. *Challenge:* a criterion strict enough to refuse correctly may be too strict to find present objects within budget. Sensors cannot prove absence (eight of the 96 AI2-THOR episodes sat below the detector’s recall floor), and objects move. *Management:* the criterion is developed on present-target arms first, and every present-target battery is a regression gate. The declared recall is itself a claim with provenance, so the checker can reject an unsupported one. The certificate carries its window, and any later movement event touching the target invalidates it. Above the proof, conformal prediction on the present-target arms calibrates the refusal threshold, the KnowNo construction [41] moved from plan options to room coverage. The false-refusal rate then carries a user-set bound as well as a certificate. The bound assumes exchangeable calibration and deployment episodes, which nightly object movement violates, so it holds per movement window and is re-calibrated when a window closes. O1 can pass this synthetic test and still fail in a deployment whose declared recall is wrong, so O3 precedes any real-world claim for O1. *Acceptance:* on the absent-target battery at bar 0.9, 0/300 wrong commitments (today 121/300) with a checked NOT-FOUND in at least 250/300, and 0/120 wrong under a three-action budget (today 19/120). The 300/300 controls, 36/36 real-pixels hunts, 83/83 AI2-THOR commitments and the 360-episode benchmark stay byte-identical, or every difference is reported with its proof.

O2. Bounded per-decision cost over an unbounded lifetime. *Task:* the two remaining maintained views, an ordered contradicted-claim index kept under status events and an event-time-sorted per-subject sighting index. If the slope is still not flat, a multi-tier ledger: a hot window of raw claims plus consolidated summaries that keep the proof provenance of every folded claim. *Challenge:* consolidation may change decisions or orphan proofs, and timing on a shared machine is noisy. *Management:* the standing equivalence gates,

record-dump SHA-256 and differential tests against the previous implementation on every call. Any changed decision is reported with both proof paths. Every increment is measured as at least five interleaved paired repetitions, led by load-independent counts. *Acceptance*: on a 1,200-day soak, first-to-last-window growth at most $2\times$ (today $4.5\text{--}8.7\times$ over 300 days), with decisions byte-identical or every difference reported.

O3. Identity without instance ids. *Task*: replace the physics-pose association stand-in of §4.4 with a real multi-object tracker with re-identification, whose identity claims enter the ledger with confidence and typed status. The real-pixels protocol gains same-class decoys placed by manifest. *Challenge*: a tracker’s binding errors are a new source of false sightings the ledger has never seen. *Management*: identity claims enter at REPORTED grade until an inspection confirms them, so a mis-binding can cost actions but not a commitment. The decoy protocol is pre-registered before the clips exist, as every sitting has been. *Acceptance*: a pre-registered sitting with same-class decoys in every zone, 0 wrong commitments, finds not below the previous sitting, and the close-call margin reported per hunt.

O4. The physical robot. *Task*: the packaged ROS 2 stack on a TurtleBot3-class platform under ROS 2 Jazzy, with the detector in a hardware Sense node. The cognition node’s coordinate discard is corrected so the planner’s cost term is live. The 21 integration checks are repeated on hardware in a mapped room, with drift and SLAM error recorded at every arrival. *Challenge*: real navigation fails in ways the simulated body did not, and detector inference costs 130–240 ms per frame on the laptop. *Management*: the executor seam already treats an aborted or timed-out goal as one unit of budget with no claim recorded and re-plans from the unchanged ledger. The four failed simulated runs before the 21/21 result are the checklist. Decision latency is measured apart from stack latency, so the ~ 5 ms figure is tested on its own terms. *Acceptance*: 21/21 on hardware, the same inspection sequences as simulation where the world matches, a false tip injected over ROS abandoned, and any wrong commitment reported with its ledger.

O5. Semantic priors without training data. *Task*: an advisory prior over where an object class tends to be, drawn from an open commonsense source or the mind’s own confirmed finds. It enters at the planner’s weight hook, which today returns 1.0 for every room unless a learned habit fires, and never overrides an observation or the exhaustion invariant. *Management*: priors enter as HYPOTHESIZED-status weights, off by default, on the template of the lifelong-mind result where a wrong habit cost -15 to $+7$ actions and no commitment. *Acceptance*: a pre-registered reduction of at least 10% in mean actions on both the 96-episode AI2-THOR battery and the 36-hunt real-pixels battery, with 0 wrong commitments. Results must be byte-identical to the baseline when the prior is off, and the threshold is set here, before the prior exists.

O6. External validity. *Task*: a pre-registered head-to-head on a third-party object-search benchmark against a third-party opponent. The candidate is the pomdp_py Multi-Object Search domain with its POUCT planner and false-positive dial, the object-oriented POMDP of Wandzel et al. [52] as implemented in Zheng and Tellex [60] and extended in Zheng et al. [61]. The comparison is on that family’s native metrics, not a forced success-rate table. For the open-vocabulary line the benchmark is SCOUT’s SymSearch [32], whose reference system also runs no language model at inference time, so the comparison isolates the decision layer. *Management*: one world and one setting first, under the benchmark-iteration protocol. The adapter may not read the target’s placement, as no adapter here may. *Acceptance*: on the pomdp_py domain, the engine’s success rate within the Wilson 95% interval of the POUCT planner’s at matched budget and false-positive rate. That arm also requires 0 wrong commitments and every proof path checked. On SymSearch, the engine’s success rate reported beside the reference system’s under the benchmark’s own protocol. Both are committed before the paper cites them, whichever way they fall. The first arm is a held-out generator family for the warehouse benchmark (location count, carrier count, report reliability and delays drawn from ranges the rules

were not designed on). It carries the same zero-wrong gate and requires success within five points of the home family at every budget, a threshold set here before the family exists.

Mechanism ablations, done. The leave-one-out control that O6 needed is in §4.1: on the shipped build, multi-anchor hypotheses and repetition decay each carry a disjoint share of the sub-exhaustive margin, and transport hand-off’s share is within noise on the home family. O6’s held-out family re-runs the same three arms, since a mechanism that is idle on the home family need not be idle on one with more carriers.

Sequencing and completion criterion. O1 comes first, since it needs no hardware and is the missing half of the innovation. O2 runs in parallel, O3 on the next real-pixels sitting, O4 when the platform is in hand, and O5 and O6 after O1. The programme is complete when O1, O2 and O4 meet their acceptance tests on committed artifacts that an outside group has regenerated. That means certified commitment and certified refusal, at bounded lifelong cost, on a physical robot. The settings that benefit are those where a wrong commitment is a safety or mission event and an audit trail is required. Hospital and laboratory logistics, warehouse and tool-crib search, disaster-response and defence-adjacent search, and planetary and underwater survey are each already modelled by a battery above. The component that benefits them is the decision layer alone. It consumes an existing stack’s detections and navigation through the ROS 2 seams of Appendix A, and is briefed through typed goals rather than trained.

9 Conclusion

An append-only, bi-temporal evidence ledger with typed statuses, replayable proof paths and a verify-then-redirect search policy is sufficient for a correct, efficient and auditable closed-domain object-search agent. On the warehouse benchmark it is 100% correct at 1.75 actions and abandons planted lies at 3.25, and under budgets brute force cannot exhaust it holds 98–100% at four and five actions where an exact belief-space planner over the same evidence reaches 73–90%. No policy, the engine included, commits wrongly in the sweep’s 18,000 episodes, because success requires a confirming inspection. Two hosted LLM agents match its accuracy and not its ~5 ms episodes, its determinism or its proofs. Unchanged, the same build scores 134/134 on ALFWorld, fetches 95/96 in AI2-THOR and scores 12/12 on SubT worlds. It passes 21/21 integration checks on stock Nav2 with SLAM Toolbox in simulation, and commits correctly in 36/36 hunts on a hand-held phone camera’s footage of a house. A mind kept across six days of nightly change holds 12/12 every day. Every one of those commitments carries a derivation record that a checker outside the engine replays from a shipped ledger export. §8 sets out the research and development that follows, with an acceptance test per objective.

Reproducibility and data availability

Three things are released. The *benchmark kit* (footnote on the title page) holds the scenario generator, the scripted and exact belief-space baselines of §4.1, the LLM comparison harness, the ROS 2 message definitions, a proof checker, a decision replayer and one verify command. A *demonstration build* of the engine ships beside it, so a reader can run the engine on scenarios of their own. Every artifact this paper reports ships too. The engine’s *source* is not released, including the computations withheld in §2. The consequence is worth stating plainly: the method is not reproducible from this paper, and the numbers and the proofs are. The contributions are therefore stated as measured behaviour, not as mechanism.

Per-episode ledger exports ship for every battery whose commitments the kit’s checker re-validates with no engine code importable: the warehouse held-out battery (`battery_final`), the budget sweeps at 12 and at

100 seeds, calibration, the six head-to-head runs, the absent-target battery, the Wizard-of-Oz house batteries (`woz_test`, `woz_selfimprove`, `woz_dynamic`, `woz_dynamic_night480`) and the unseen-seed audit. That is 10,008 engine episodes and 228,989 proof paths. The remaining evaluations ship as per-episode artifacts rather than ledger exports: ALFWorld, AI2-THOR (whose placements are not bit-stable), SubT, the hospital battery, Mars and seabed, the ROS 2 lanes, the phone battery, the stress battery, the self-repair exams and the edge profile. What the checker certifies is provenance: every cited claim and rule exists and the stated time constraints hold on replay. It does not certify the conclusion, which is graded against hidden truth instead.

A few figures in this paper come from earlier builds and are labelled where they appear. The head-to-head’s original measurement session and the dynamic-house collapse counts are that build’s record and are artifact-logged; the shipped build’s own figures are the ones the kit regenerates. Every number regenerates from integer seeds except the AI2-THOR episodes, which are artifact-logged because the simulator’s placement randomization is not bit-stable. Summary evaluation JSONs ship in `data/artifacts/`, and `docs/EMBODIED.md` carries per-episode tables, stand-up notes and reproduction commands for the embodied evaluations. The real-pixels battery ships its pre-registered manifests, per-episode artifacts and protocol with every sitting’s result (`docs/PHONE_PROTOCOL.md`). Its clips are available on the same request basis as the bundle. `scripts/reproduce.py` runs the native suites in one command, compares every regenerated number with the committed artifact, and restores the committed files. Its report ends with one gaslight episode traced end to end and its counterfactual. These engine-side commands run in the author’s environment. `docs/REPRODUCTION.md` carries the per-suite command block, the regenerate-to-verify list and the artifact bundle’s contents with its manifest and SHA-256 sums. Appendix A is the integration note for a ROS 2 stack. A pre-submission audit recomputed every artifact-backed number from the 2026-09-01 shipped JSONs, and all agreed (the soak growth ratios in §4.7 and the staleness re-rank counts in limitation 4 are quoted from the author’s stress-test and design notes rather than from a shipped JSON); the regenerate-to-verify figures are listed in `docs/REPRODUCTION.md`.

Broader impact statement

The engine is an auditable alternative to LLM decision-making for search-and-localization tasks. Its proof paths and counterfactual queries are designed to make automated decisions contestable, which we view as a positive contribution to accountability in deployed autonomy. Two points of scope apply. One evaluation uses the DARPA Subterranean Challenge’s virtual worlds. Subterranean search is dual-use (disaster response and military reconnaissance alike), but nothing here advances the hard parts of fielding such systems: perception, locomotion and communications. We evaluate a search policy on public competition worlds under public scoring. The hospital-logistics benchmark involves no patient data or clinical decisions. It is a fetch task on a synthetic floor. The engine’s determinism and offline operation reduce data-exfiltration and prompt-injection surfaces relative to LLM-agent deployments. Its guarantees hold at its typed input boundary, whose failure modes §4.7 stress-tests and reports.

Acknowledgments

The system, its benchmarks and this paper were produced by an LLM assistant, Anthropic’s Claude, under the author’s direction. The author supplied goals, evaluation standards and review (limitation 7), has reviewed the paper, and takes full responsibility for its content and claims. The audits and red-team reviews cited throughout were internal: separate sessions of the same assistant, whose findings, including defects in earlier results, are preserved in the repository.

Funding and competing interests

This work received no external funding. The author conducted it as an independent researcher. The author is the owner and chief executive of ModelBrew.ai, which had no role in this work and whose products are unrelated to it.

A Dropping the engine into a ROS 2 stack

This appendix is for a robotics engineer who wants to know what the engine assumes, what an adapter has to supply, and what a minimal integration looks like. The ROS 2 package in `ros2/` is the blueprint; the simulator adapters in `experiments/embodied/` are the same contract without ROS.

What the engine assumes. Three things, and nothing about the platform. (1) A closed ontology: the entities the robot can reason about (locations, containers, persons, objects) are registered by id and type before or during an episode; the engine never invents one from free text. (2) Typed claims: every input is a claim $\langle \text{subject, predicate, object, confidence, source, event time, ingestion time} \rangle$ with a predicate from a fixed list (`LOCATED_AT`, `ABSENT_FROM`, `INSIDE`, `NEAR`, `ENTERED`, `MOVED_TO`, ...). Raw sensor data never enters; a perception module’s job is to emit claims at the confidence it actually has. (3) A small action vocabulary the platform can execute: inspect a location, scan a container, pick up, and the sense call that returns what is visible from the current pose. The engine decides which of these to do next, when a sighting settles the find, and when to give up; it does not drive, map, grasp, or see.

What an adapter supplies. A location map (ids and, on a robot, poses); a *sense* function that returns what is visible from the current pose (in the packaged node, entity ids, the sensor’s confidence in each, and the nearest registered location; the node mints a `LOCATED_AT` claim per sighting at the confidence the sensor reported, 0.95 when a logical sensor reports none, and, when the target is not among them, an `ABSENT_FROM` claim at 0.90, a fixed sensor-model constant of that node; a detector’s own confidence therefore reaches the ledger through the *Sense* answer itself, through *IngestClaim*, or through the vision seam, and a find is settled only when the engine’s own verdict resolves the target where the robot stands); and an *executor* for the action vocabulary that moves the platform and calls *sense*. The adapter is forbidden to read the target’s placement and is graded by the platform’s own signals; every adapter in this paper obeys that rule and `docs/EMBODIED.md` records each one’s sensing model beside its score. Teleport locomotion and simulator-state sensing in the AI2-THOR and gz-sim evaluations are properties of those adapters, chosen to isolate the search policy; they are not requirements of the engine, and the ROS 2 stack replaces both with a navigation server and a sensor node without touching the engine.

The ROS 2 shape. One managed lifecycle node, `cognition_node`, wraps the unchanged `EpisodeRunner`: `configure` reads parameters and registers locations, `activate` accepts goals, `deactivate` refuses them. It exposes one action and three services, and is itself a client of whatever `NavigateToPose` server the platform runs (Nav2’s `bt_navigator` on a robot; a demo driver in the bare sim):

action	/cognition/find_object	robot_mind_msgs/action/FindObject
srv	/cognition/ingest_claim	robot_mind_msgs/srv/IngestClaim
srv	/cognition/query_belief	robot_mind_msgs/srv/QueryBelief
client	/cognition/sense	robot_mind_msgs/srv/Sense
client	/navigate_to_pose	nav2_msgs/action/NavigateToPose

`FindObject` takes a target (entity id or class), a visit budget and whether a grasp pose is wanted, and returns found, the resolved entity and location, the inspections spent, the proof path, and the pose for a

manipulation pipeline; its feedback carries the current status, location and best hypothesis at every step. IngestClaim is the tips protocol over ROS: one typed claim into the live episode's ledger, from any source (a second perception stack, a human), queued by the service thread and drained by the episode thread between actions because the ledger's SQLite connection is thread-bound. QueryBelief is the audit read: the resolved location, ranked hypotheses with their belief breakdowns, and the missing variables the engine would seek next.

A minimal integration. Parameters of the cognition node, then one goal:

```
cognition_node:
  ros__parameters:
    robot_mind_src: /opt/robot_mind/src # the engine, imported unchanged
    location_names: [bay_1, bay_2, bay_3, bay_4]
    location_x:     [ 6.0,  6.0, -6.0, -6.0]
    location_y:     [ 6.0, -6.0,  6.0, -6.0]
    view_dist:      2.2 # stand-off from a location, metres
    goal_frame:     map # odom for the demo driver, map on Nav2

ros2 action send_goal /cognition/find_object \
  robot_mind_msgs/action/FindObject \
  "{target: target_box, max_visits: 12, want_grasp_pose: true}"
```

A sensor node answers Sense with the entity ids it sees and its confidence in each: the logical frustum sensor of the first two lanes (3.0 m, 0.7 rad half-angle, a frustum test over bridged physics poses, no confidence field), or the camera lane's yolo_sense_node (YOLO-World on the bridged RGB frame, detections bound to entities by projecting their poses through the camera intrinsics, the detector's score returned per id); on a robot it is the platform's detector and tracker, through the same seam the AI2-THOR real-detector battery exercises (ingestion/vision_adapter.py: a DetectionEvent with label, camera id, confidence and time becomes an OBSERVED claim at the detector's confidence, and the association between detections and entities is the tracker, whose failure modes the engine's verify loop exists to catch). Nothing else changes between the demo driver, stock Nav2 in simulation and the camera lane; the same client would run the same list on hardware. The integration checks are one list (20 checks; 21 with Nav2 readiness; 26 with the camera lane's detector facts, a frame per inspection, two boxes in view, verdict-gated confirmations and no wrong-instance commitment), and the numbers in §4.4 are what it read.

The Nav2 stack behind the 21/21, and one caveat. Lane two of §4.4 ran stock Nav2 1.3.12 (bt_navigator, the NavFn planner, the DWB controller, the behavior server and the velocity smoother), with localization from SLAM Toolbox over a roof lidar. One caveat bounds the 21/21. Under Nav2 the frustum sensor (3.0 m range, 0.7 rad half-angle, 2.2 m stand-off) looks from the robot body's physics pose, a stand-in for a localized robot. The search is what is graded, and the localization columns of Table 10 say what the stand-in cost. A sensor at the SLAM pose would have stood within 9 cm of the vantage used, against a 3 m sensing range. Nav2's parameters were changed only for this body and arena. The inventory, the plugin names read from the running servers, and the four bring-up runs are in Appendix B and in ros2/README.md.

Operating the engine. An operator audits a decision through the append-only ledger, proof replay by the checker, the QueryBelief query over ROS 2 and the whatif counterfactual re-run. Each returns the following. The append-only ledger holds every claim with its source, two clocks and its status history. Over ROS 2, QueryBelief returns the resolved location, the ranked hypotheses with their belief breakdowns, and the missing variables the engine would seek next. The whatif command re-runs an episode with a claim

removed and reports what changed. The `FindObject` action’s feedback carries the current best hypothesis and status at every step. Uncertainty is signalled as the ranked beam and its missing variables rather than a single answer. Learned rules reach deployment only through the candidate $\rightarrow$ provisional $\rightarrow$ validated lifecycle, with held-out precision recorded, and even then act as advisory priors that an observation overrides. No path lets a learned rule create an OBSERVED fact.

What is not built. A human-in-the-loop review step for rule promotion, a signalling policy for a particular high-stakes setting, MoveIt 2 beyond the grasp-pose hand-off, and any run on hardware.

B Development record: what was tried, what failed, what changed

This appendix carries the chronology that the main text summarises: the development history of each battery, the runs that failed and what was changed, with every number that left the main text.

Architecture: module inventory and later organs

§2 describes only the loop that runs on every decision; this block lists the modules and the organs added after v0.

Modules. The single Python application (74 source files) has ten modules. Five handle input and memory: ingestion and normalization, entity and time grounding, the SQLite evidence ledger, the belief-graph view over trusted claims, and salience. Grounding uses weighted identity that preserves ambiguity below threshold, and salience detects contradictions and gaps. Five reason and act: the hypothesis engine, the multi-hop reasoner, the information-gain planner, the simulated-warehouse executor with verified postconditions, and the evaluator and learner. Version v0 shipped the deterministic template implementation of the `CognitiveProposer` protocol, with no model dependency.

Organs of a persistent mind. Later versions added temporal decay of evidence, run-compression of repeated observations, drives with utility arbitration, staleness-sized patrols, declarative plans with fallbacks and a calibration meter. A metacognitive layer keeps expectation records that settle every organ’s predictions, diagnoses typed violations, and drives the gated self-repair loop of §4.7.

The learned rule and the three guarantees of a valid proof

The transport rule of §4.1 is mined from the engine’s own episodes with support 450, training precision 0.71 and held-out precision 0.77; its deployed reliability is 0.71, the lower of the two. It is promoted through a candidate $\rightarrow$ provisional $\rightarrow$ validated lifecycle and deployed only after validation, after which proof paths cite the learned rule and its measured reliability. The contextual bandit over action types is trained with ϵ -greedy exploration and evaluated frozen. The phrase “valid proof” covers three separate guarantees, of which the paper certifies the first in full. *Provenance integrity*: every cited claim and rule exists in the ledger and the stated time constraints hold. The checker verifies this by replay, and it is 1.000 in every reported run. *Logical consistency*: the conclusion follows from the declared rules. The checker confirms the rule ids and their arity but does not re-derive the conclusion, so this is checked structurally, not semantically. *Empirical correctness*: the conclusion matches independent ground truth. This is measured separately, as the correct-commitment rates reported with each result, and never inferred from proof validity. A valid trail says the decision was made from these inputs by these rules. It never says the inputs were true.

Scripted baseline definitions

The scripted policies of Table 5 run 12 seeds per level on levels 2–6, under the same contract as the LLM agents: same generator, executor, 8-action budget, success rule and hidden-truth grading. The *sweep* checks all locations, then all carts, in sorted order, and ignores the evidence. The *random-order* walk visits the same places in a seeded random order. The *last-seen-first* heuristic checks the target’s most recent sighting, then sweeps. The *Bayesian belief filter* keeps a posterior over holders (locations, carts, persons) and checks the argmax. It has the belief updating any POMDP-lite tracker provides, given the same evidence, and none of the engine’s machinery: no ledger, no epistemic statuses, no contradiction bookkeeping, no proof paths, no multi-anchor hypotheses, no transport handoff. Scripted baselines were run before the engine’s number was read, because they show what a benchmark measures; the ALFWorld ablation of §4.3 is the case in point. On the gaslight level the random order cannot be deceived and has no fixed order to be unlucky in, which is why it pays the least of the scripted policies. Naive evidence use is worse than no evidence use, and disciplined evidence use beats both. The LLM rows of Table 7 read in this light: the 7–8B models fail episodes that a for-loop passes, so their deficit is sustained systematicity, not search skill.

Budget sweep configuration

The budget sweep changes one field of the benchmark’s configuration (`experiments/budget_sweep.py`). The action budget is the only change; generator, executor, success rule and grading are those of the LLM contract. At eight actions every policy is at 100%, and the 12-seed subset of the sweep reproduces the scripted and planner rows of Table 5. The engine row differs by build: 2.10 on the current build against the original campaign’s 2.05 on L2–6, as the caption of Table 6 states. Zero wrong commitments is a structural property of the success rule, shared by the random walk; what a smaller budget removes is the chance to inspect everything. Two shapes in Figure 2 carry the mechanism. The random walk climbs steadily with budget, as coverage should. The belief filter stalls at 42–44% from four actions to six, because argmax-posterior over the same evidence locks onto the wrong branch of a transport chain and keeps confirming it. The engine’s multi-anchor hypotheses and transport handoff turn that same evidence into the right first look.

The exact belief-space planner

The planner of §4.1 is solved to optimality rather than approximated, because the benchmark’s search problem has a closed structure. The world is static during a search. A check is a Bernoulli sensor with a known hit rate (0.97 inspect, 0.98 scan, the executor’s own constants) over the holders it can reveal. A hit ends the episode, and the Bayes updates for misses commute. The finite-horizon value $V(S, k) = \max_{a \notin S} [p_a(b_S) + (1 - p_a(b_S))V(S \cup \{a\}, k - 1)]$ is therefore a dynamic programme over subsets of the eight-action menu (`experiments/belief_planner.py`). It starts from the belief filter’s reading of the briefing evidence and uses the true sensor model, so it is the optimal policy for the POMDP the filter’s model defines. At the full budget it finds everything in 3.12 actions to the filter’s 4.80. Per level at four actions: on the gaslight level the planner finds 37% and the engine 95%; on levels 4 and 5 (stale claims and decoys; contradictions and occlusion) the planner finds 73% and 57% against the engine’s 98% and 99%; on the carrier and cart-chain levels both find everything. The filter finds 42.6% at the same budget.

Calibration methodology

The calibration numbers of §4.1 pool every final-beam hypothesis against hidden truth, 25 seeds per level. Levels 1–5 give ECE 0.047 (Brier 0.097); the gaslight level gives ECE 0.065 (Brier 0.116). The reliability diagrams (Figure 3) show a conservative signature: beliefs above 0.7 are empirically right 100% of the time,

and the only material overconfidence sits in the 0.1–0.3 bins. Adversarial evidence degrades calibration slightly and never inverts it.

Transfer ladder and Wizard-of-Oz ladder

Beyond the static benchmark, a transfer ladder drove the engine’s growth: each rung exposed a missing capability, each capability was built as a general organ, and the warehouse battery was held at 100% throughout. A home world (dishwashers that hide items, wandering people) tests transfer: find one item 0.96; find all five 0.88; find, collect and brew in dependency order 0.88 (25 seeds per rung). The rung-3 procedure also runs as data, a declarative plan with dependency edges and fallback branches interpreted by a generic plan runner, with measured pass/fail parity against the hand-coded sequencer on every seed (`battery_plans.json`). Procedurally generated houses entered with an empty ledger failed at first (W2 5/25, W3 0/25); the missing capability was an exploration drive. The engine’s governor clustered its failures into machine-readable capability requests, the requested organs were built, and the battery re-measured W2 0.20→0.88 and W3 0.00→0.68. The v0.3 memory organs (age-decayed evidence) lifted them to 1.00 and 0.96.

Re-taking the same house daily with memory persisting separates two claims. On the pre-drive engine the ten-attempt curve was flat, 4/15 on every attempt, a deterministic fixpoint: a house whose rooms were all known was still never entered beyond its last sighting, because nothing connected an unseen goal to an unvisited room. Tuning choices among conceivable actions cannot conceive new ones. The drive and fallback organs did, and the battery’s self-improvement test rose from 0.25 to 0.875 with them and to 1.00 with the memory organs, passing on the first attempt (attempt 1 = attempt 6, slope 0; `woz_selfimprove.json`, `battery_final.json`).

The dynamic house

The dynamic-house protocol separates memory from learning: twelve houses, six days each, 1–2 items moved nightly, the mind persisting. The pre-v0.3 engine scored 12/12 on day 1 and 4, 1, 0, 2 and 0 of 12 on days 2–6 with drives off (6, 5, 4, 2 and 3 of 12 with drives on). With per-episode budgets, refutation-driven beam widening and age-decayed evidence the same mind recovers to 12/12 every day. With realistic 8-hour nights the drive-less mind erodes to 4/12 by day 6 while the patrolling mind holds 10/12, and staleness-sized patrols drop patrol spend from 5.8 to 2.6 actions/day at identical success. Persistent memory without forgetting was a liability in a changing world; decay plus refutation-driven search turned it into an asset.

Head-to-head: the reasoning model, the scaffold, the adversary and the hybrid seam

These details from §4.2 do not change its numbers. Two local models were run: an older instruct-only 7B and a newer ~8B reasoning model whose explicit chain of thought is the belief tracking at issue. The reasoning model scores 45/48 against the 7B’s 40/48 at roughly 8× the latency, so the accuracy gap narrows as the model is allowed to reason. The strong scaffold’s strategy is: enumerate competing explanations, including “the cart left empty”; choose the discriminating test; never repeat an empty check. Under it the 7B model lifted L3 to 12/12 but lost ground at L5, and its failures still concentrate on the deceptive branches. The model can recite the hypothesis list and still follow the salient cart trail into a dead end, because nothing in its context durably tracks what its own failed checks have ruled out.

The L6 adversary’s further moves, beyond the confident fake sighting, are sometimes a second contradictory sighting, a staged same-type twin object at the fake location, and occasionally a false absence report at the true location. The engine’s multi-anchor hypothesis generation keeps the true carrier chain alive

regardless of what the liar says. In the `CognitiveProposer` seam, ambiguous names in a natural request are resolved against the entity registry rather than guessed.

ALFWorld: what the benchmark tests, and the seen-split defect

The scripted ablations of §4.3 support one reading of the benchmark, and the second run followed one engine fix. ALFWorld’s difficulty for LLM agents is sustained systematicity, durable state tracking and never rechecking. A ledger-based mind, a for-loop and a random walk all possess those, and current LLM agents do not. The seen-split defect was a “ghost anchor”: after the first instance of a two-object task was collected, a lingering class-level sighting let the refuted-fallback resurrect already-checked rooms. The fix is belief hygiene. The stale sighting is retired through the ledger’s own append-only supersede mechanism, and the give-up condition was corrected. Both were developed and validated on the seen split alone (134→138/140) before the unseen split was run a second time.

AI2-THOR: planner, verification seeds and the fix campaign

The 95/96 battery of §4.3 carried two verification seeds, and its planner has a classical form. The next-best-view planner is active perception in its classic set-cover formulation [12, 42]. Seeds 137 and 314 were never used to diagnose a fix; 137 ran 12/12, and the campaign’s one miss fell on 314. The engine was untouched throughout. The planner’s construction and the adapter-side fix campaign, including the two vantage strategies that were tested and rejected, are documented in `docs/EMBODIED.md`. The text-world result of §4.3 carries over to the 3D and physics simulators it abstracts.

SubT: stack status and visit-count audit

Three facts behind the 12/12 SubT result. The worlds load content-unmodified in gz-sim Harmonic, headless and CPU-only. The competition’s cloudsim stack is end-of-life, so what runs is DARPA’s worlds and DARPA’s scoring rule, not the competition infrastructure. An internal audit re-derived every visit count from world geometry alone, and the per-episode counts (2–87 of a 250-visit budget) match that derivation.

Hospital campaign: protocol rules, seeds, artifacts, reproduction and the adapter fix

These are the protocol rules and run-level facts behind the hospital numbers of §4.3. In the plain fetch protocol, targets come only from classes with one instance in the world, since a resident sibling would satisfy a class-level find while grading demands the relocated item. Two episodes drew truth spots inside the first visit’s sensing bubble, the deepest episode worked through 21 of the 102 spots, and per-episode visit counts keep that 1–21 spread visible. The evaluator restores world state after each episode, so the simulator is as fresh per episode as the engine’s ledger.

In the planted-tip protocol the tip, when false, names a decoy spot. The three deciders share one bridge, budget and grading, and the naive tip-follower visits the tip spot first and then falls back to sweep order. The standard eight seeds drew six true and two false tips. The four balancing seeds were chosen after that split was known, by their precomputed tip coin alone and before any of them ran; the eight-seed and four-seed artifacts ship separately. Every policy’s 12/12 follows from the protocol, which guarantees it for any complete searcher that commits only on sightings. The engine’s true-tip 1.00 visits is six times fewer than the sweep’s 6.17. The tip-follower’s true-tip 1.00 is its policy by construction; the engine’s matching 1.00 is measured behavior, the reported spot ranked first in all six episodes. The false-tip permutation result ($p = 0.91$ against the sweep, $p = 1.0$ against the tip-follower) has a plain cause: one debunked tip is too little evidence structure for belief arithmetic to beat a fixed fallback order. The claims that carry weight are the true-tip collapse to 1.00 visits, the zero wrong commitments, and the ledger trail that shows each debunking. The protocol is the

warehouse verify-then-redirect pattern (§4.1) reproduced in a physics simulator on the AWS RoboMaker hospital world’s floor plan.

In the decoy round the look-alike is a spawned twin instance. The class-matching tip-follower’s five-of-six wrong commitments come at one visit each, fast and wrong, and the sweep is fooled in four of six because its fixed route reaches the staged twin before the true item. The engine is not fooled because the look-alike’s sighting enters the ledger under its own instance id, the same read records the named target’s absence, the tip’s belief collapses, and the search redirects. That identity check is not a bolted-on rule; it is carried in the entity bookkeeping. The thirty-seed power run takes the first thirty integer seeds whose tip coin is false, declared in docs/EMBODIED.md before running. Its artifact (hospital_tips_twin_30.json) was assembled from three launches after two Gazebo transport stalls, with every launch and log recorded there. Of the six seeds shared with the six-episode round, four reproduced the engine’s visit count and two moved: seed 3 from 22 to 36 visits and seed 5 from 24 to 25, while every scripted arm reproduced. The code path is identical. The likeliest cause is that the engine arm runs first after the target and the look-alike are placed, and sensed before the placed bodies had settled in physics. The counts are reported as measured.

Two facts about the campaign build. A probe of the slowest false-tip episode found the adapter under-reporting its own sensor: absence was recorded for the visited spot alone, at lower confidence than the positive channel of the same read. With the sensor reported faithfully that episode fell from 96 to 24 visits. The whole hospital campaign, plain fetch included, was re-run on the fixed build, so every hospital number in the paper and in Table 8 is post-fix. The engine also re-verifies a debunked tip up to three times across its seek restarts, because repetition-decay state resets per seek and, with no positive evidence anywhere, the debunked tip briefly returns to the argmax. Those checks are counted in the 14.67 and cost no commitment.

Mars and seabed: worlds, placements, visit counts and what is not claimed

The two field-geometry batteries of §4.3 are specified here in full. The Space ROS Mars rover demonstration’s Curiosity terrain runs from the demo packages built on ROS 2 Jazzy with gz-sim Harmonic. The terrain is a 304 m square with 28 m of relief, of which a 72 m square is gridded into 169 survey stations at a 6 m pitch, sensed by a logical 8 m mast sensor. Project DAVE’s ocean-objects world holds 11 unique-instance seabed objects within 26 models, a flight data recorder, unexploded ordnance and hard hats among them, over 70 survey stations. Its sensor is a BlueView P900 sonar footprint with a declared range-to-confidence curve and fan-shaped absence bookkeeping. Placements are pre-registered per seed and confirmed by reading the entity state back before the first sense, and truth is the station nearest the read-back pose.

Visit counts, four seeds per arm with engine, sweep and random-order arms sharing the executor, budget and sensor. Mars: 4/4 correct in every arm, engine 21–58 visits with mean 31.0, sweep 45.0, random 26.0. Seabed: 4/4 in every arm, engine 2–12 visits with mean 7.75, sweep 21.25, random 4.25. Zero wrong commitments in all 24 episodes (Wilson 95% [0.51, 1.00] per arm). Until evidence arrives the engine’s order is a nearest-first sweep, which is why a random order can beat it by luck at $n = 4$. The planted-tip and staged-decoy variants for both worlds are pre-registered with tip coins drawn and seeds fixed, and have not been run. Not claimed: driving the rover or the vehicle, sonar perception, Space ROS itself (the demo packages run on vanilla Jazzy), or lunar worlds. HoloOcean is blocked on WSL2 and Stonefish is the viable fallback (docs/SPACE_MARINE.md).

The AI2-THOR detector battery: history, misses and operating point

This block carries the development history, the per-miss detector scores and the operating point of the real-detector battery in §4.4.

- *Fresh draws.* AI2-THOR redraws placements per launch. The eight seeds are therefore fresh draws of each configuration, not replays of the oracle-sensing episodes.

- *First campaign at 480².* A seed-7 smoke went from 5/12 to 10/12 with six adapter-side, target-blind fixes. The campaign scored 27/36 on seeds 7, 11 and 23. Rendering at 1080² recovered three of the remaining episodes. The confirmation-ordering fix recovered a fourth.
- *Which seed produced which fix.* The synonym table was built from seed-7 misses. The confirmation-ordering fix was diagnosed on seed 11.
- *The one adapter defect.* The campaign exposed an ordering bug in the session’s give-up test. It was fixed so that a sighting between the claim gate and the confirmation bar resolves by the engine’s own verdict. The engine itself was not touched.
- *Crash reruns.* Two held-out episodes (FloorPlan203 on seeds 42 and 137) crashed at simulator start-up. The cause was a Unity initialisation timeout under GPU contention, before a target had been chosen. Each was rerun alone as a fresh draw. Both fetched. The crash records ship in the artifacts, marked superseded.
- *Misses below the detector’s recall (eight).* Two forks and a butter knife inside a garbage can. A keychain inside a can. Three credit cards lying flat. An alarm clock on a shelf that the detector localized at IoU 0.97 but never named above 0.10.
- *Mislabels (four).* Each was localized at IoU 0.84–0.93 but named as something else. A jar candle read as “Cup” at 0.54. A pepper shaker read as “SoapBottle” at 0.92. A butter knife on a dining table read as “Pen” at 0.27. A plunger read as “Spatula” or “BaseballBat” at 0.15. None was added to the synonym table, because each is a single spawn under the two-spawn rule.
- *Never rendered (one).* A CD inside a box on a bed, a container the adapter does not open.
- *Probing.* Every miss was probed from the receptacle’s planned vantages and from approach poses (`scripts/thor_detector_probe.py`). Each was reported not-found after the full budget (`docs/EMBODIED.md`).
- *Operating point.* Weights `yolov8x-worldv2`. Inference at `imgsz 1088` on the 1080² render. A raw detector floor of 0.02, so that weak candidates reach the associator. Association by $\text{IoU} \geq 0.35$ against the instance’s own rendered box. The label must match the instance’s class or one of eleven fixed synonym groups: Bottle/WineBottle/SoapBottle/SprayBottle/WateringCan, Cup/Mug, Towel/HandTowel/Cloth, Pot/Pan/Kettle/Bowl, Pen/Pencil, DishSponge/SoapBar/Cloth/Book, and five more in `adapters/thor_yolo.py`. A 0.05 confidence gate at the ledger. The detector is prompted once per process with the 62-type vocabulary.
- *Cost.* A fetched episode consumed a median of 10 detector frames (1,766 over the 36 episodes). Inference costs about 42 ms per frame on the laptop GPU used (`docs/EMBODIED.md`). The software-rendered simulator costs seconds per frame. The detector is therefore not where the 259 s mean wall time per fetched episode goes.
- *The seam’s event type.* The `DetectionEvent` the seam accepts carries a label, camera id, confidence, time and an optional `track_id`. On a platform with a tracker that field is filled. The track is then linked to an entity through the same weighted resolver the warehouse benchmark uses, and the association step described above does not exist. That configuration is the next build (Table 13, last row).

ROS 2 bring-up: bridge runs, the packaged node, Nav2 parameters, plugin names, the four diagnostic runs, and the camera lane's sensor node

This block is the bring-up record behind the ROS 2 lanes of §4.4; `ros2/README.md` annotates every parameter change.

- *Bridge runs before the packaged node.* A logical-camera scout hunted a relocated box over the `gz.transport13` bridge to Gazebo Harmonic, 4/4 episodes correct. The same loop then ran over ROS 2 topics (Jazzy, `ros_gz`). It drove a diff-drive base through `/cmd_vel` and odometry with no teleports, 4/4.
- *The packaged node.* Beside the `FindObject` action and the `NavigateToPose` client, the node exposes claim-ingest and belief-query services. The false tip of the demo-driver lane was injected mid-episode through the ingest service. It was verified and abandoned over ROS: bay_4, then bay_3, then bay_4 re-verified, then bay_2, the truth. Under uniform priors the node inspects in the fixed order bay_4, bay_3, bay_2, bay_1, so the find counts of 2, 4, 1 and 3 inspections are the truth's position in that sweep, and the tip is the discriminating lane.
- *Parameters.* The DWB local planner runs at 0.8 m/s and 0.5 rad/s, with 0.5 m/s² and 0.5 rad/s² ramps and a 0.6 velocity-smoother backstop. The ramps keep the sphere-wheeled body's odometry accurate, because ramped commands do not slip the wheels. MPPI at the Jazzy default crawled at 0.09 m/s on this body. Regulated Pure Pursuit stalled on the 180° bay-to-bay turns once angular acceleration was capped. NavFn plans over a global costmap built from live scans rather than the SLAM map, with 0.1 m costmaps and a 0.25 m goal tolerance. SLAM Toolbox adds a graph node per 1.0 m or 0.5 rad and publishes a 0.1 m map every 5 s. Transform tolerance is 0.5 s throughout. Wall time per inspection is almost all driving between bay stand-off poses.
- *Plugin names.* The artifacts name the plugins as read from the running servers (`nav2_facts: dwb_core::DWBLocalPlanner`, `nav2_navfn_planner::NavfnPlanner`, Nav2 1.3.12). The Nav2-lane artifact's controller query timed out during bring-up. It records an empty `controller_plugin`, with `controller_server` (unknown) in `nav2_nodes`, rather than a typed-in name. The sensor vantage, `sensor_vantage: physics_pose`, is read from the running sensor node in the same way.
- *Worlds.* The artifacts were regenerated on the final world, whose bay boxes are the cardboard cubes of the camera lane. The first run of the checks, on 1.0 m posts, read 0.093 m of odometry drift and 0.047 m of SLAM error with the same inspection sequences. The demo-driver checks, regenerated on the same final world, pass 20/20 with identical sequences.
- *Four diagnostic runs.* Bringing Nav2 up took four runs. Each was resolved in the simulated body or in Nav2/SLAM configuration, and none in cognition: odometry-frame sensing, an axle/origin offset, an intermittent boot failure, and SLAM starving the navigator. Run 4 is the other end of the navigation-failure path. With SLAM starving the navigator, eight navigation attempts, five of them aborting within 60 ms, ended in not-found rather than a guess.
- *The camera lane's sensor node.* The `yolo_sense_node` runs YOLO-World on each frame with a fixed generic vocabulary ("cardboard box", "crate", "barrel", "pillar", ...; never an entity id). It binds each box detection to a known entity by projecting that entity's physics pose through the camera intrinsics (IoU ≥ 0.35 , each entity bound once). The AI2-THOR battery's 0.02 floor, 0.05 claim gate and 0.35 gate are untouched for this world. The node answers with the entity ids and the detector's own confidence. The cognition node writes that number into the `LOCATED_AT` claim. Detector time was measured on the laptop GPU, and the annotated frames ship in the artifact bundle.

- *The camera lane’s confirmation rule.* The episode runner settles a find only on an observation at ≥ 0.9 . A detector score of 0.3–0.6 never is, and must not be inflated to it. The executor therefore writes the raw sighting at the detector’s number. It adds a single inspection-confirmation claim (0.97, provenance confirmed_by: engine_verdict and detector_confidence) only when the engine’s own verdict resolves the target at the bay the robot is standing at. The artifact lists every such confirmation with the verdict that opened it.
- *The camera-lane timeout.* In the tip episode of the YOLO-World lane, Nav2 reported “Failed to make progress” on the 135° reversal out of the bay_2 stand-off. It cleared its costmap and recovered, and the retry outran the node’s 120 s result wait. The engine took the failed action and re-planned, which is what reordered the inspections. The decisions were the same as under the frustum sensor: verify the tip twice, then redirect.

The phone battery: sittings one to four

This block records the four phone sittings of §4.4 in order, with the three fixes made between the first graded sitting and the pre-registered fourth.

1. *Sitting one.* Filmed off its manifest. Kept as a pilot and not graded.
2. *Sitting two, the first graded.* Pre-registered as a manifest with the engine and sense model fixed before the clips existed (docs/PHONE_PROTOCOL.md). The engine scored 12/36, level with the sweep (12/36) and the random order (13/36). Raw detector output had found the one place where the loop’s resolve-on-sighting rule is wrong. The rule is right wherever an inspection can confirm a sighting. A clip cannot, because the same clip returns the same detection. Three causes were fixed. First, the phone adapter now sets a sighting bar of 0.9, the loop’s own confirmation confidence; the engine default is unchanged and every native suite byte-identical. Below the bar a sighting is a hypothesis, the search goes on, and an exhausted search commits to its uniquely best-supported zone. Second, a coverage-bookkeeping fault in the harness’s search loop. Third, a sense-model vocabulary (aliases, distractor prompts) for a detector prompted with household words. Re-run on the same clips, the fixed engine scored 29/36. Every one of the seven wrong is a perception error named in the artifact. A power bank and a photo frame read as the phone. A white mug went unseen on white marble.
3. *Sitting three.* Pre-registered on the fixed engine. It scored 33/36, with 15/18 true tips followed and 18/18 false tips debunked. The three wrong were again perception. The remote read as a phone above a phone lying on a dark bench without its placemat. The tumbler read as a soap bottle.
4. *Sense model v3.* One label per object across every class, with synonym folding, on the same weights and prompt list. It was chosen on the graded footage of sittings two and three (29/36 unchanged; 33/36 to 35/36), then pre-registered for sitting four.
5. *Sitting four.* One run, nothing changed after seeing it: 36/36 finds, 18/18 true tips followed, 18/18 false tips debunked (§4.4).

The lifelong curve: two protocol facts

Two protocol facts bound the reading of the 95-to-10 habit result in §4.5.

- The seeded coin drew habit episodes for all ten post-validation episodes. The validated habit’s off-habit cost therefore rests on the four-episode wrong-habit run (+1, −13, +7, −15 actions).

- A first curve on this protocol validated the habit and saved nothing, because the class rule had been instantiated on a single countertop. The defect was fixed before the reported curve. Both curves ship.

Self-repair exams v1 and v2

This block carries the two sealed fault-injection exams behind the self-repair paragraph of §4.7, and the review that turned the first into the second.

sabotage	damage	diagnosis	repair adopted	recovery
memory decay constant cut 40×	0.965 → 0.256	SEARCH_EXHAUSTION	sabotaged field, restored	100%
eyes tp 0.97 → 0.01	0.965 → 0.790	FRUITLESS_ACTIONS	sabotaged field, restored	100%
scanner tp 0.98 → 0.01	0.965 → 0.026	FRUITLESS_ACTIONS	sabotaged field, restored	100%
repetition-decay constant 4× weaker	0.965 → 0.826	(none — below threshold)	(none, correctly)	0%
nothing (healthy)	—	(none)	(none, correctly)	—

Table 14: Self-repair exam v1: the three detectable single-field sabotages are diagnosed and repaired to 100% recovery, the sub-threshold sabotage and the healthy control are correctly left untouched, and the placebo passes in 0% of raced lanes. “tp” is a sensor’s true-positive rate.

Table 14 gives exam v1. Exam v2 jitters every numeric field (46 differ from the archive) before one is sabotaged hard, and adoption uses only the engine’s own success signal. Both diff-defeating lanes recover 100%/98% by touching only the sabotaged field among 46 candidates. A no-archive lane recovers 99% by pure factor-climbing, and the drift-only control rests with zero false repairs. One lane fails: the no-archive scanner catastrophe stays broken, because greedy one-edit repair cannot cross score-flat valleys without an archive. Sabotages come from the v1 detectable set, so the capability shown is recovery of pre-screened, detectable, single-field faults under drift. Self-repair here means configuration repair behind an expectation monitor, with four ceilings (limitation 9). The loop cannot invent a missing representation, fix a wrong objective, repair its own diagnostic machinery, or see a failure it holds no expectation about.

The review of exam v1, and the lanes that adopted nothing. These are the reasons exam v2 was built, and what the lanes that adopted nothing, or adopted the wrong thing, did. An internal review of exam v1 (Table 14), run as a separate session of the same assistant, raised four objections. The sabotage roster was screened for detectability. A one-line config diff against the archive would solve every v1 lane. The v1 candidate repairs were selected by the exam’s hidden-truth score. The violation vocabulary names ~25% of the numeric config surface. Exam v2 was built to answer them by construction (§4.7). In exam v1 the sub-threshold lane weakened the repetition-decay constant fourfold, which cut success from 0.965 to 0.826. That damage sits under the violation threshold, so no violation was raised, no repair was proposed, and recovery is 0%, the correct outcome. The healthy control likewise produced no violation and no repair. In exam v2 the failing no-archive scanner lane did adopt one non-sabotaged-field edit that passed the gates without helping, recorded in its artifact. The 120-corruption sweep of the stress battery (§4.7) shows the loop’s restraint at scale.

The absent-target battery: verification of the manipulation, gate counts and review history

This block carries the detail that left §4.6, with every number. The sweep labels every arm with the bar it ran at and every verdict with the path, direct or exhaustion, that produced it. The absence manipulation was verified rather than assumed: every write path into world state was enumerated, world truth reads *unknown* in 300/300 episodes, no episode materialises the target at run time over 1,800 episodes, and the seed streams are byte-identical to the controls. In the absent arm at bar 0.9 the weak-sighting branch of the instrumented

gate is entered 1,210 times across the arm’s 3,000 reasoning calls. The exhaustion verdict fires 799 times; 678 of those are retracted by the inspection they trigger, and 121 are still held when the budget ends. In the length-matched control at full budget the direct path names no wrong room, and with a strong sighting in the ledger the exhaustion rule never decides the episode.

Three separate-session reviews of the harness preceded acceptance of its numbers. Each began from the assumption that the headline bucket was unreachable by construction; that failure mode, a headline bucket no episode could reach, had occurred twice earlier in the project. The accepted figures were re-derived by a classifier route that never touches the harness’s reporting code. Of the two diagnostic arms, the bar-0.0 absent arm shows 36/300 transient direct-path verdicts and 0/300 held at the end. They rest on a level-5 low-confidence camera artefact (0.45–0.62) that passes the direct path’s freshness test while the confidence test is vacuous at a bar of zero. They occur in the same episodes where the present-target control holds a transient verdict, and the first inspection retracts them, so the 0/300 is an anchor-freshness window and not restraint. The evidence-stripped arm returns the same uniform belief vector in 300/300 episodes, a wiring check that no verdict is invented from nothing.

Detail moved from the numbered limitations

Each paragraph below carries the material cut from the limitation it names, with every number.

Limitation 4, the probes behind rule set v0.2.0. The probes that produced the current rule set (v0.2.0) were hand-built temporal scenarios run after the first campaign. They exposed two defects the benchmark had not surfaced: rooms never seen could outrank the last-seen room on staleness alone, and containers could not be located. Both were fixed before the budget sweep and the planner race. Original-campaign and post-fix engine figures are both reported (1.75 and 1.80 actions on L2–5).

Limitation 8, the Wilson 95% intervals. The intervals quoted in the table captions: 95/96 $\rightarrow$ [0.94, 1.0]; 12/12 $\rightarrow$ [0.76, 1.0]; 8/8 $\rightarrow$ [0.68, 1.0]; 83/96 $\rightarrow$ [0.78, 0.92]; 6/6 $\rightarrow$ [0.61, 1.0]; 20/20 checks $\rightarrow$ [0.84, 1.0]. The embodied intervals are wide at these sample sizes.

Limitation 8, the decoy power run. The six-seed decoy round is directional rather than powered: 0 of 6 fooled has a Wilson upper bound of 0.39, which excludes a policy fooled most of the time but not one fooled a third of the time. The 30-seed powered run used the first thirty integer seeds whose tip coin is false. It was declared in docs/EMBODIED.md before running, and its launches are recorded there with their logs. Two launches aborted, one by a process-cleanup pattern and one by machine load. A third and a fourth completed eleven and eighteen seeds before stalling on a Gazebo transport fault, and their rows were recovered from the console logs. A fifth ran the last seed. The pre-registered thirty seeds give 0/30 fooled (Wilson upper bound 0.11) at 11.7 visits (§4.3).

Limitation 8, seed-family variance. One visible instance: the learned build’s 1.47 actions on the held-out family 10000–10074 against 1.80 on the budget-sweep family 20000–20099, both on levels 2–5. Variance across seed families is otherwise unquantified and single-machine.

Limitation 10, SLAM and the SubT score. SLAM runs only in the Nav2 integration checks, where the sensor vantage still comes from physics. In the SubT protocol the position half of the scoring rule is inherited from the sensor by construction.

Limitation 12, the two real components and the real-pixels sitting. The evaluations invest in simulator fidelity, a real detector on the perception seam, a real navigation stack on the navigation seam, and artifact reproducibility. The detector ran in AI2-THOR and Nav2 in the gz-sim arena. The combined lane, YOLO-World in the Sense slot of the Nav2 robot, passed 26/26 checks in a four-bay arena with same-mesh boxes and an association step that knows the entities’ physics poses, a stand-in for a tracker. The real-pixels battery removes the simulator and the association step from sensing but not the robot: the camera was carried by hand, the six zones were declared rather than mapped, and locomotion was a person walking. Its 36 hunts are one house, one camera and one sitting, so its interval is wide (Wilson lower bound 0.90), and one of the 36 was decided by a belief margin of 0.02 over a look-alike reading. A simulator is also where hidden truth, stale and adversarial claims, action budgets, thousands of seeds and exact replay of every failure can be controlled at once, which a single physical platform cannot offer.

C Glossary

- ECE: the expected calibration error, the mean absolute gap between stated belief and observed frequency over confidence bins.
- Brier score: the mean squared error of the stated belief.
- IoU: intersection over union between a detector box and an instance mask.
- NBV: next-best view, the vantage that uncovers the most unseen volume.
- POMDP: partially observable Markov decision process.
- POUCT: partially observable UCT, the sampled planner `pomdp_py` ships.
- SLAM: simultaneous localization and mapping.
- ROS 2: the Robot Operating System 2. Nav2 is its navigation stack, whose DWB and MPPI controllers are its dynamic-window and model-predictive path-integral local planners.
- WSL 2: the Windows Subsystem for Linux.
- EGL: the headless OpenGL interface some simulators require.
- EOL: end of life.
- `tp`: a sensor’s true-positive rate, in the self-repair tables.

References

- [1] Ashutosh Adhikari, Xingdi Yuan, Marc-Alexandre Côté, Mikuláš Zelinka, Marc-Antoine Rondeau, Romain Laroche, Pascal Poupart, Jian Tang, Adam Trischler, and William L. Hamilton. Learning dynamic belief graphs to generalize on text-based games. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2020. arXiv:2002.09127.
- [2] Michael Ahn, Anthony Brohan, Noah Brown, Yevgen Chebotar, Omar Cortes, Byron David, Chelsea Finn, et al. Do as I can, not as I say: Grounding language in robotic affordances. *arXiv preprint arXiv:2204.01691*, 2022.
- [3] John R. Anderson, Daniel Bothell, Michael D. Byrne, Scott Douglass, Christian Lebiere, and Yulin Qin. An integrated theory of the mind. *Psychological Review*, 111(4):1036–1060, 2004.

- [4] Francesco Argenziano, John Mark Alexis Marcelo, Michele Brienza, Abdel Hakim Drid, Emanuele Musumeci, Daniele Nardi, Domenico D. Bloisi, and Vincenzo Suriani. R2F: Repurposing ray frontiers for LLM-free object navigation. *arXiv preprint arXiv:2603.08475*, 2026.
- [5] Dhruv Batra, Aaron Gokaslan, Aniruddha Kembhavi, Oleksandr Maksymets, Roozbeh Mottaghi, Manolis Savva, Alexander Toshev, and Erik Wijmans. Objectnav revisited: On evaluation of embodied agents navigating to objects. *arXiv preprint arXiv:2006.13171*, 2020.
- [6] Thomas Bolander and Mikkel Birkegaard Andersen. Epistemic planning for single- and multi-agent systems. *Journal of Applied Non-Classical Logics*, 21(1):9–34, 2011. doi: 10.3166/jancl.21.9-34.
- [7] Peter Buneman, Sanjeev Khanna, and Wang-Chiew Tan. Why and where: A characterization of data provenance. In *Database Theory — ICDT 2001*, Lecture Notes in Computer Science, pages 316–330, 2001. doi: 10.1007/3-540-44503-X_20.
- [8] Minghao Chen, Yihang Li, Yanting Yang, Shiyu Yu, Binbin Lin, and Xiaofei He. Automanual: Constructing instruction manuals by LLM agents via interactive environmental learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2024. arXiv:2405.16247.
- [9] Tianheng Cheng, Lin Song, Yixiao Ge, Wenyu Liu, Xinggang Wang, and Ying Shan. YOLO-World: Real-time open-vocabulary object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2024.
- [10] C. Chow. On optimum recognition error and reject tradeoff. *IEEE Transactions on Information Theory*, 16(1): 41–46, 1970. doi: 10.1109/TIT.1970.1054406.
- [11] Timothy H. Chung, Viktor Orekhov, and Angela Maio. Into the robotic depths: Analysis and insights from the DARPA subterranean challenge. *Annual Review of Control, Robotics, and Autonomous Systems*, 6:477–502, 2023.
- [12] C. Connolly. The determination of next best views. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 432–435, 1985.
- [13] Michael T. Cox. Metacognition in computation: A selected research review. *Artificial Intelligence*, 169(2): 104–141, 2005.
- [14] Johan de Kleer. An assumption-based TMS. *Artificial Intelligence*, 28(2):127–162, 1986.
- [15] DeepSeek-AI. DeepSeek-V3 technical report. *arXiv preprint arXiv:2412.19437*, 2024.
- [16] Jon Doyle. A truth maintenance system. *Artificial Intelligence*, 12(3):231–272, 1979.
- [17] Salomé Eriksson, Gabriele Röger, and Malte Helmert. Unsolvability certificates for classical planning. In *Proceedings of the International Conference on Automated Planning and Scheduling (ICAPS)*, volume 27, 2017. doi: 10.1609/icaps.v27i1.13818.
- [18] Yonatan Geifman and Ran El-Yaniv. Selective classification for deep neural networks. In *Advances in Neural Information Processing Systems (NeurIPS)*, volume 30, 2017. arXiv:1705.08500.
- [19] Google DeepMind. Gemini api model documentation: gemini-3.7-flash. <https://ai.google.dev/gemini-api/docs/models>, 2026. Accessed August 2026.
- [20] Todd J. Green, Grigoris Karvounarakis, and Val Tannen. Provenance semirings. In *Proceedings of the 26th ACM SIGMOD-SIGACT-SIGART Symposium on Principles of Database Systems (PODS)*, pages 31–40, 2007. doi: 10.1145/1265530.1265535.
- [21] Alasdair Hill, Ekaterina Komendantskaya, and Ronald P. A. Petrick. Proof-carrying plans: a resource logic for AI planning. In *Proceedings of the 22nd International Symposium on Principles and Practice of Declarative Programming (PPDP)*, pages 1–13, 2020. doi: 10.1145/3414080.3414094. arXiv:2008.04165.

- [22] International Organization for Standardization. ISO 13482:2014 — robots and robotic devices — safety requirements for personal care robots. Geneva, Switzerland, 2014. <https://www.iso.org/standard/53820.html>.
- [23] Leslie Pack Kaelbling and Tomás Lozano-Pérez. Integrated task and motion planning in belief space. *The International Journal of Robotics Research*, 32(9–10):1194–1227, 2013.
- [24] Jeffrey O. Kephart and David M. Chess. The vision of autonomic computing. *IEEE Computer*, 36(1):41–50, 2003.
- [25] Eric Kolve, Roozbeh Mottaghi, Winson Han, Eli VanderBilt, Luca Weihs, Alvaro Herrasti, Matt Deitke, Kiana Ehsani, Daniel Gordon, Yuke Zhu, et al. AI2-THOR: An interactive 3D environment for visual AI. *arXiv preprint arXiv:1712.05474*, 2017.
- [26] John E. Laird. *The Soar Cognitive Architecture*. MIT Press, 2012.
- [27] Claire Le Goues, ThanhVu Nguyen, Stephanie Forrest, and Westley Weimer. GenProg: A generic method for automatic software repair. *IEEE Transactions on Software Engineering*, 38(1):54–72, 2012.
- [28] Bo Liu, Yuqian Jiang, Xiaohan Zhang, Qiang Liu, Shiqi Zhang, Joydeep Biswas, and Peter Stone. LLM+P: Empowering large language models with optimal planning proficiency. *arXiv preprint arXiv:2304.11477*, 2023.
- [29] Peiqi Liu, Yaswanth Orru, Jay Vakil, Chris Paxton, Nur Muhammad Mahi Shafiullah, and Lerrel Pinto. OK-Robot: What really matters in integrating open-knowledge models for robotics. *arXiv preprint arXiv:2401.12202*, 2024.
- [30] Joel Loo, Zhanxin Wu, and David Hsu. Open scene graphs for open-world object-goal navigation. *arXiv preprint arXiv:2508.04678*, 2025.
- [31] Matt Luckcuck, Marie Farrell, Louise Dennis, Clare Dixon, and Michael Fisher. Formal specification and verification of autonomous robotic systems: A survey. *ACM Computing Surveys*, 52(5), 2019. arXiv:1807.00048.
- [32] Imen Mahdi, Matteo Cassinelli, Fabien Despinoy, Tim Welschehold, and Abhinav Valada. Relational semantic reasoning on 3D scene graphs for open world interactive object search. *arXiv preprint arXiv:2603.05642*, 2026.
- [33] Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. *arXiv preprint arXiv:2310.08560*, 2023.
- [34] Yu Qian, Hong Miao, Boyang Guo, Tingyi Jiang, Shan Zhao, Tianxing Le, Lintian Li, and Meng Liu. ECHO: A cognitively inspired, auditable memory plane for long-horizon agents. *arXiv preprint arXiv:2608.21755*, 2026.
- [35] Libin Qiu, Zhirong Gao, Junfu Chen, Yuhang Ye, Liangyu Li, Weizhi Huang, Xiaobo Xue, Wenkai Qiu, and Shuo Tang. Autorefine: Compiling trajectories into validated typed agent artifacts. *arXiv preprint arXiv:2601.22758*, 2026. Reports 100.0% (134/134) single-attempt success on ALFWorld with a GPT-5.6-terra backbone.
- [36] Qwen Team. Qwen2.5 technical report. *arXiv preprint arXiv:2412.15115*, 2024.
- [37] Qwen Team. Qwen3 technical report. *arXiv preprint arXiv:2505.09388*, 2025.
- [38] Krishan Rana, Jesse Haviland, Sourav Garg, Jad Abou-Chakra, Ian Reid, and Niko Sünderhauf. SayPlan: Grounding large language models using 3D scene graphs for scalable robot task planning. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.06135.
- [39] Preston Rasmussen, Pavlo Paliychuk, Travis Beauvais, Jack Ryan, and Daniel Chalef. Zep: A temporal knowledge graph architecture for agent memory. *arXiv preprint arXiv:2501.13956*, 2025.
- [40] Donald B. Reid. An algorithm for tracking multiple targets. *IEEE Transactions on Automatic Control*, 24(6): 843–854, 1979.
- [41] Allen Z. Ren, Anushri Dixit, Alexandra Bodrova, Sumeet Singh, Stephen Tu, Noah Brown, Peng Xu, Leila Takayama, Fei Xia, Jake Varley, Zhenjia Xu, Dorsa Sadigh, Andy Zeng, and Anirudha Majumdar. Robots that ask for help: Uncertainty alignment for large language model planners. In *Conference on Robot Learning (CoRL)*, 2023. arXiv:2307.01928.

- [42] William R. Scott, Gerhard Roth, and Jean-François Rivest. View planning for automated three-dimensional object reconstruction and inspection. *ACM Computing Surveys*, 35(1):64–96, 2003.
- [43] Hardik Shah, Erica Tevere, Deegan Atha, Marcel Kaufmann, Shehryar Khattak, Manthan Patel, Marco Hutter, Jonas Frey, and Patrick Spieler. WildOS: Open-vocabulary object search in the wild. *arXiv preprint arXiv:2602.19308*, 2026.
- [44] Noah Shinn, Federico Cassano, Edward Berman, Ashwin Gopinath, Karthik Narasimhan, and Shunyu Yao. Reflexion: Language agents with verbal reinforcement learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, 2023. arXiv:2303.11366.
- [45] Mohit Shridhar, Jesse Thomason, Daniel Gordon, Yonatan Bisk, Winson Han, Roozbeh Mottaghi, Luke Zettlemoyer, and Dieter Fox. ALFRED: A benchmark for interpreting grounded instructions for everyday tasks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, 2020. arXiv:1912.01734.
- [46] Mohit Shridhar, Xingdi Yuan, Marc-Alexandre Côté, Yonatan Bisk, Adam Trischler, and Matthew Hausknecht. ALFWorld: Aligning text and embodied environments for interactive learning. In *International Conference on Learning Representations (ICLR)*, 2021. arXiv:2010.03768.
- [47] Chan Hee Song, Jiaman Wu, Clayton Washington, Brian M. Sadler, Wei-Lun Chao, and Yu Su. LLM-Planner: Few-shot grounded planning for embodied agents with large language models. In *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2023. arXiv:2212.04088.
- [48] David Smith Sundarsingh, Yifei Li, Tianji Tang, George J. Pappas, Nikolay Atanasov, and Yiannis Kantaros. Safe planning in unknown environments using conformalized semantic maps. *arXiv preprint arXiv:2509.25124*, 2025.
- [49] Wenjing Tang, Xinyu He, Yongxi Huang, Yunxiao Xiao, Cewu Lu, and Panpan Cai. Tru-POMDP: Task planning under uncertainty via tree of hypotheses and open-ended POMDPs. *arXiv preprint arXiv:2506.02860*, 2025.
- [50] Cagri Temel. Towards trustworthy autonomous robots: An explainable AI-based decision framework. *arXiv preprint arXiv:2609.02861*, 2026.
- [51] Marco Tranzatto, Takahiro Miki, Mihir Dharmadhikari, Lukas Bernreiter, Mihir Kulkarni, Frank Mascarich, Olov Andersson, Shehryar Khattak, Marco Hutter, Roland Siegwart, and Kostas Alexis. CERBERUS in the DARPA subterranean challenge. *Science Robotics*, 7(66):eabp9742, 2022.
- [52] Arthur Wandzel, Yoonseon Oh, Michael Fishman, Nishanth Kumar, Lawson L. S. Wong, and Stefanie Tellex. Multi-object search using object-oriented POMDPs. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2019. doi: 10.1109/ICRA.2019.8793888.
- [53] Yuchen Xiao, Sammie Katt, Andreas ten Pas, Shengjian Chen, and Christopher Amato. Online planning for target object search in clutter under partial observability. In *IEEE International Conference on Robotics and Automation (ICRA)*, pages 8241–8247, 2019.
- [54] Shuhan Xue, Zixin Ding, Yichen Shen, Yinjie Wang, Zhenfei Yin, Yingcheng Wu, Yuxin Chen, Mengdi Wang, and Ling Yang. PAST-Bench: Benchmarking the foundations of recursive self-improvement in personal agents. *arXiv preprint arXiv:2608.04003*, 2026.
- [55] Shunyu Yao, Jeffrey Zhao, Dian Yu, Nan Du, Izhak Shafran, Karthik Narasimhan, and Yuan Cao. ReAct: Synergizing reasoning and acting in language models. In *International Conference on Learning Representations (ICLR)*, 2023. arXiv:2210.03629.
- [56] Naoki Yokoyama, Sehoon Ha, Dhruv Batra, Jiuguang Wang, and Bernadette Bucher. VLFM: Vision-language frontier maps for zero-shot semantic navigation. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2024. arXiv:2312.03275.

- [57] Zhaochen Yu, Yingcheng Wu, Zhenfei Yin, Kaiyuan Chen, Zhe Zhao, Mengdi Wang, Shuicheng Yan, and Ling Yang. Recursive experiential-working memory evolution for long-horizon agent harnesses. *arXiv preprint arXiv:2608.24876*, 2026. Introduces the Recuris system.
- [58] Jenny Zhang, Shengran Hu, Cong Lu, Robert Lange, and Jeff Clune. Darwin gödel machine: Open-ended evolution of self-improving agents. *arXiv preprint arXiv:2505.22954*, 2025.
- [59] Guoyang Zhao, Yudong Li, Weiqing Qi, Kai Zhang, Bonan Liu, Kai Chen, Haoang Li, and Jun Ma. Decision-driven semantic object exploration for legged robots via confidence-calibrated perception and topological subgoal selection. *arXiv preprint arXiv:2509.20739*, 2025.
- [60] Kaiyu Zheng and Stefanie Tellex. pomdp_py: A framework to build and solve POMDP problems. *arXiv preprint arXiv:2004.10099*, 2020.
- [61] Kaiyu Zheng, Yoonchang Sung, George Konidaris, and Stefanie Tellex. Multi-resolution POMDP planning for multi-object search in 3D. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2021. arXiv:2005.02878.
- [62] Kaiyu Zheng, Rohan Chitnis, Yoonchang Sung, George Konidaris, and Stefanie Tellex. Towards optimal correlational object search. In *IEEE International Conference on Robotics and Automation (ICRA)*, 2022. arXiv:2110.09991.